上一讲课内容回顾

- 分组转发
- 无类别路由**CIDR**
- 网络地址转换
  - ✓ ARP: IP地址 → MAC地址
  - ✓ RARP: MAC地址 → IP地址
  - ✓ NAT: 私有IP地址 → 公有IP地址
  - ✓ DNS: 域名 → IP地址

- 通过到目前为止的**课程学习**和**课程设计**，同学们应该可以回答下列问题：

① 如何**组网**？

② 网络内部是**如何实现**的？**如何工作**的？

① 如何组网？

□ 组网需要考虑哪几方面内容？

□ 需要购买哪些联网设备？设备之间如何连接？

□ 需要配置哪些服务器？对应的服务器软件

□ 网络接入

□ 各种联网设备需要的层次。

□ 网络地址与层次结构（报头）之间的对应关系

② 网络内部是如何实现的？如何工作的？

6.3节

路由

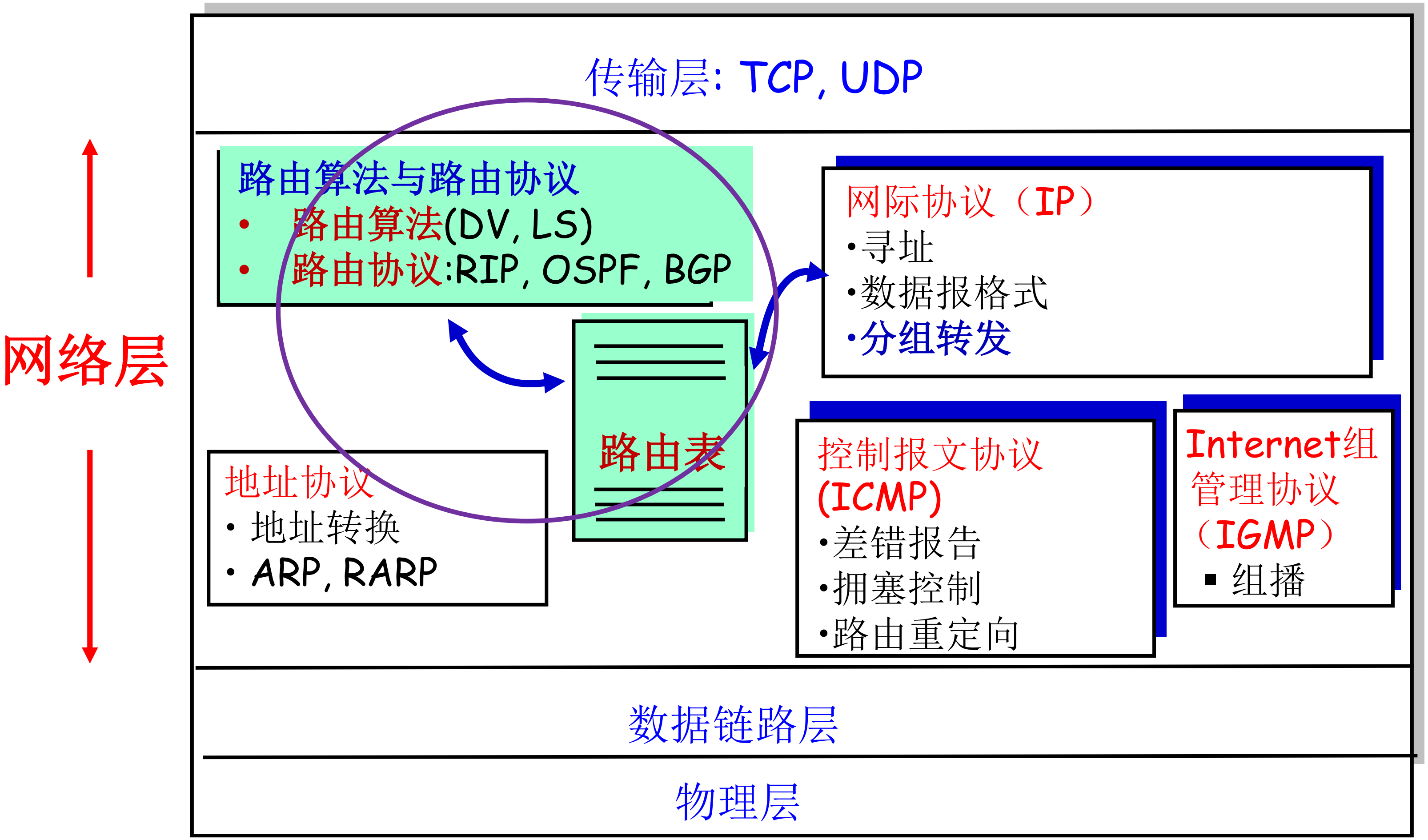
Routing

谭献海

Email：xhtan@home.swjtu.edu.cn

Internet路由相关模块

网络层功能:



传输层: TCP, UDP

网络层

路由算法与路由协议
- 路由算法(DV, LS)
- 路由协议:RIP, OSPF, BGP

路由表

网际协议（IP）
- 寻址
- 数据报格式
- 分组转发

地址协议
- 地址转换
- ARP, RARP

控制报文协议(ICMP)
- 差错报告
- 拥塞控制
- 路由重定向

Internet组管理协议（IGMP）
- 组播

数据链路层

物理层

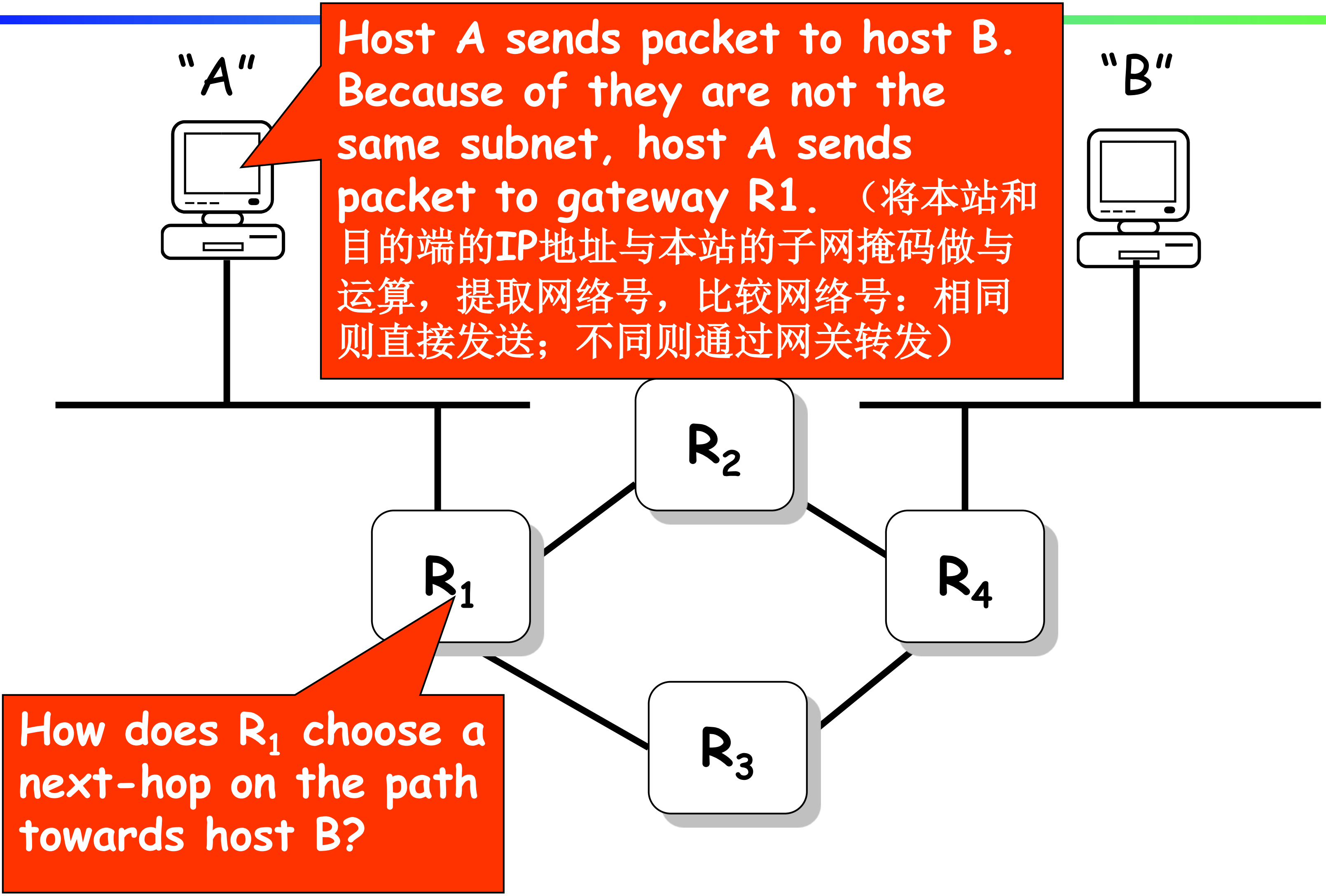
本节主要内容

- 路由的基本概念
- 路由策略
- 固定路由
- 最短路径选择算法（选路）
- 路由算法（交换路由信息，更新路由表）
- 路由协议（路由算法的具体实现）

路由的相关概念

- **路由**（Route）：从源节点到达目的节点的一条**通路(path)**
- 路由器依据什么转发分组？——**路由表**（Routing Table）
- **路由表**：保存到达目的网络的路由信息
  - 包含目的网络地址、传输路径（下一跳）、传输开销等
- 根据目的网络地址查找路由表以确定分组转发路径的过程称为**路由选择**（Routing）。
- 路由表需要随网络状态的变化而动态更新—通过**路由算法**（Routing Algorithm）来建立、更新、维护路由表：
  - 网络中的每个路由器都会根据路由算法定时或在网络状态发生变化时与其它路由器**交换路由信息**，以便更新路由表；
  - **静态路由**：预先设置路由信息，一旦设置后就不再改变；
  - **动态路由**：由路由器在运行过程中动态更新。
- 路由算法的实现就是**路由协议**（Routing Protocol）。

Routing Problem

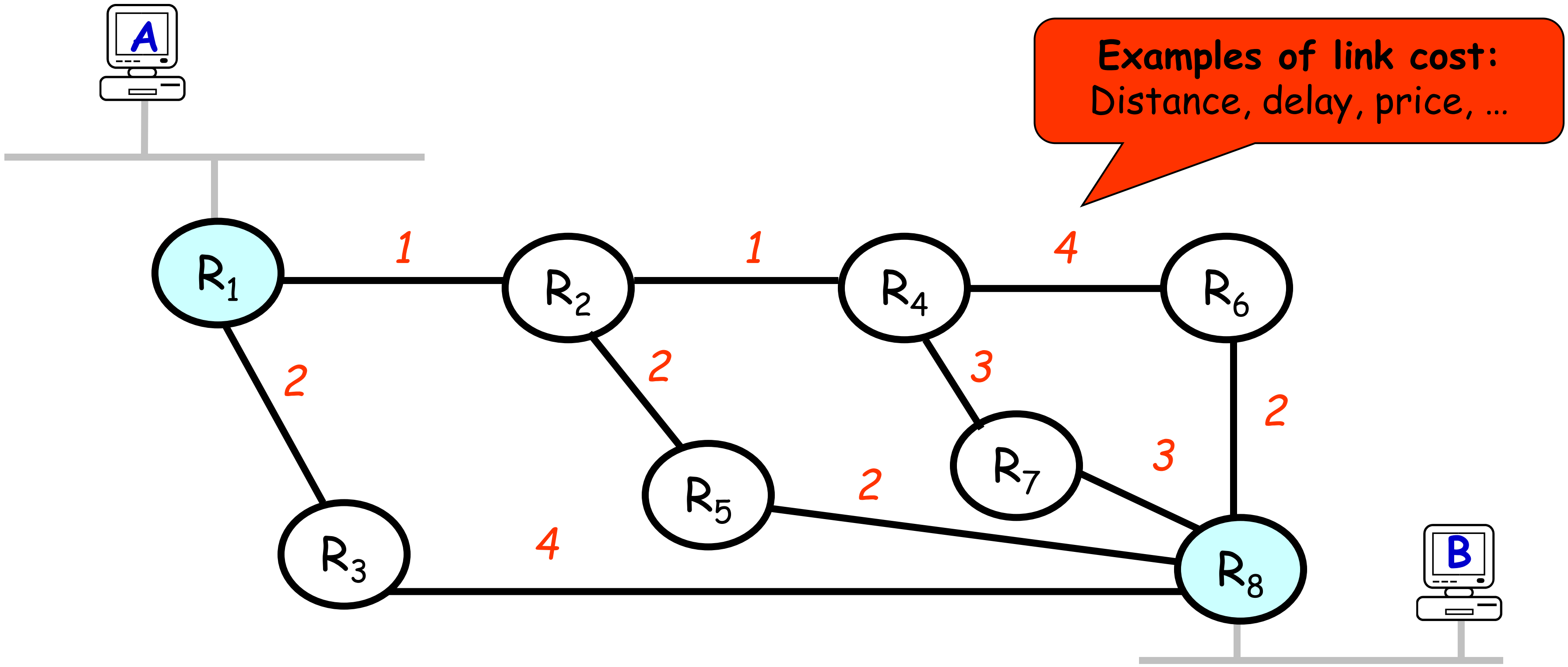


Host A sends packet to host B. Because of they are not the same subnet, host A sends packet to gateway R1. （将本站和目的端的IP地址与本站的子网掩码做与运算，提取网络号，比较网络号：相同则直接发送；不同则通过网关转发）

How does R_1 choose a next-hop on the path towards host B?

Example network

Objective: Determine the route from A to B that minimizes the path cost.

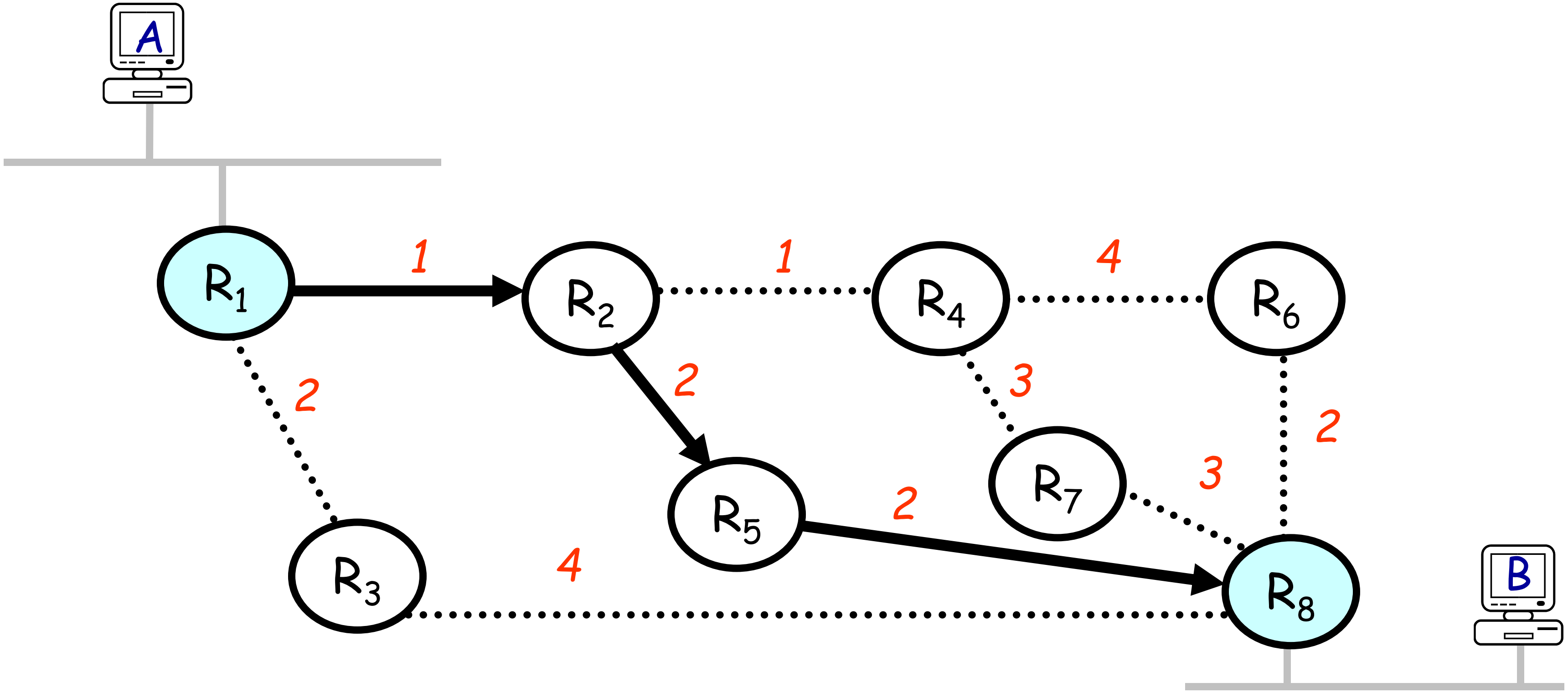

A
R_1
1
R_2
1
R_4
4
R_6
2
2
3
2
R_7
3
R_5
2
R_3
4
R_8
B

Examples of link cost:
Distance, delay, price, …

Example network

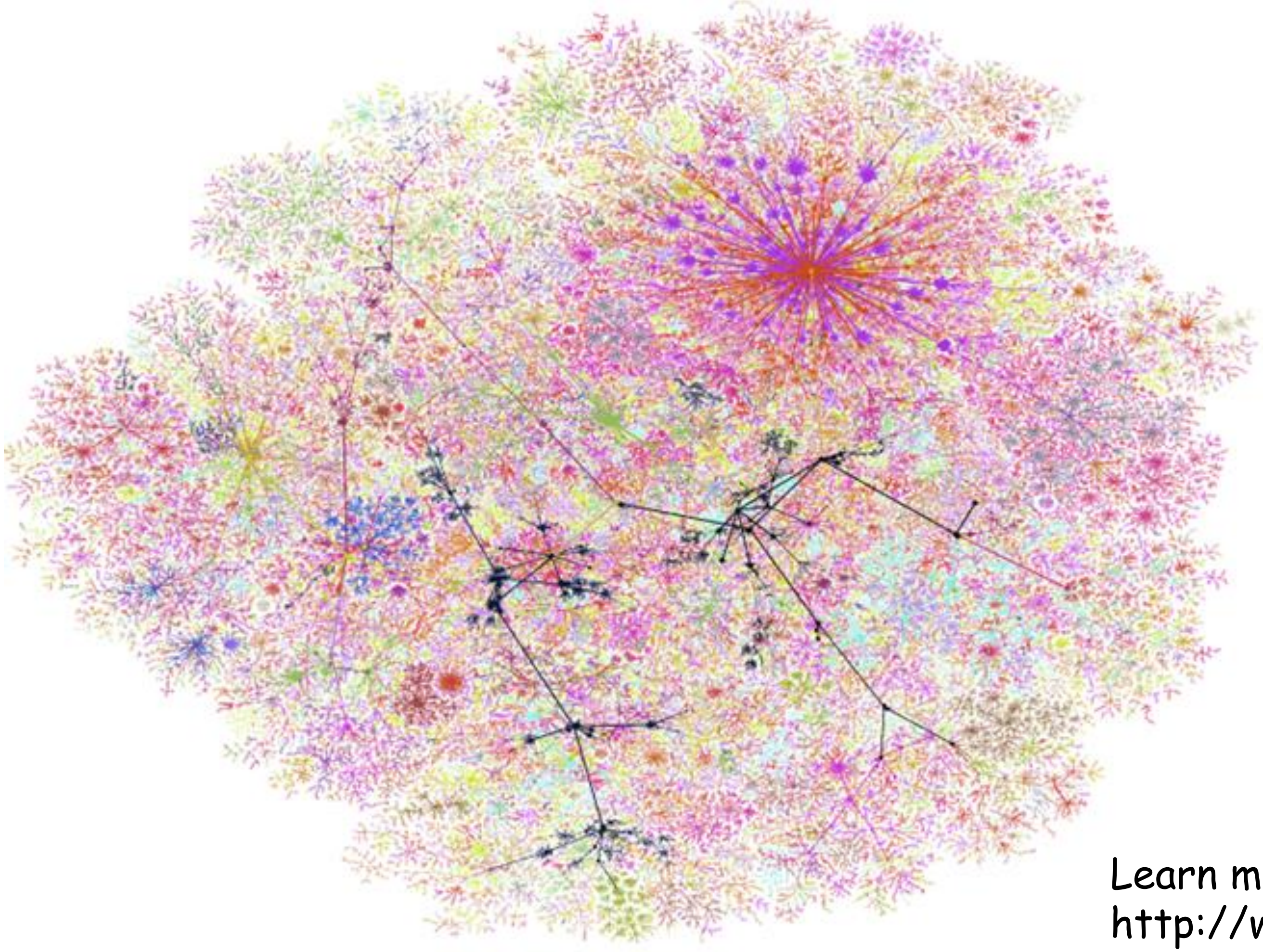
In this simple case, solution is clear from inspection

---- shortest path algorithm


A
R1
R2
R3
R4
R5
R6
R7
R8
B
1
1
4
2
2
3
2
3
2
4

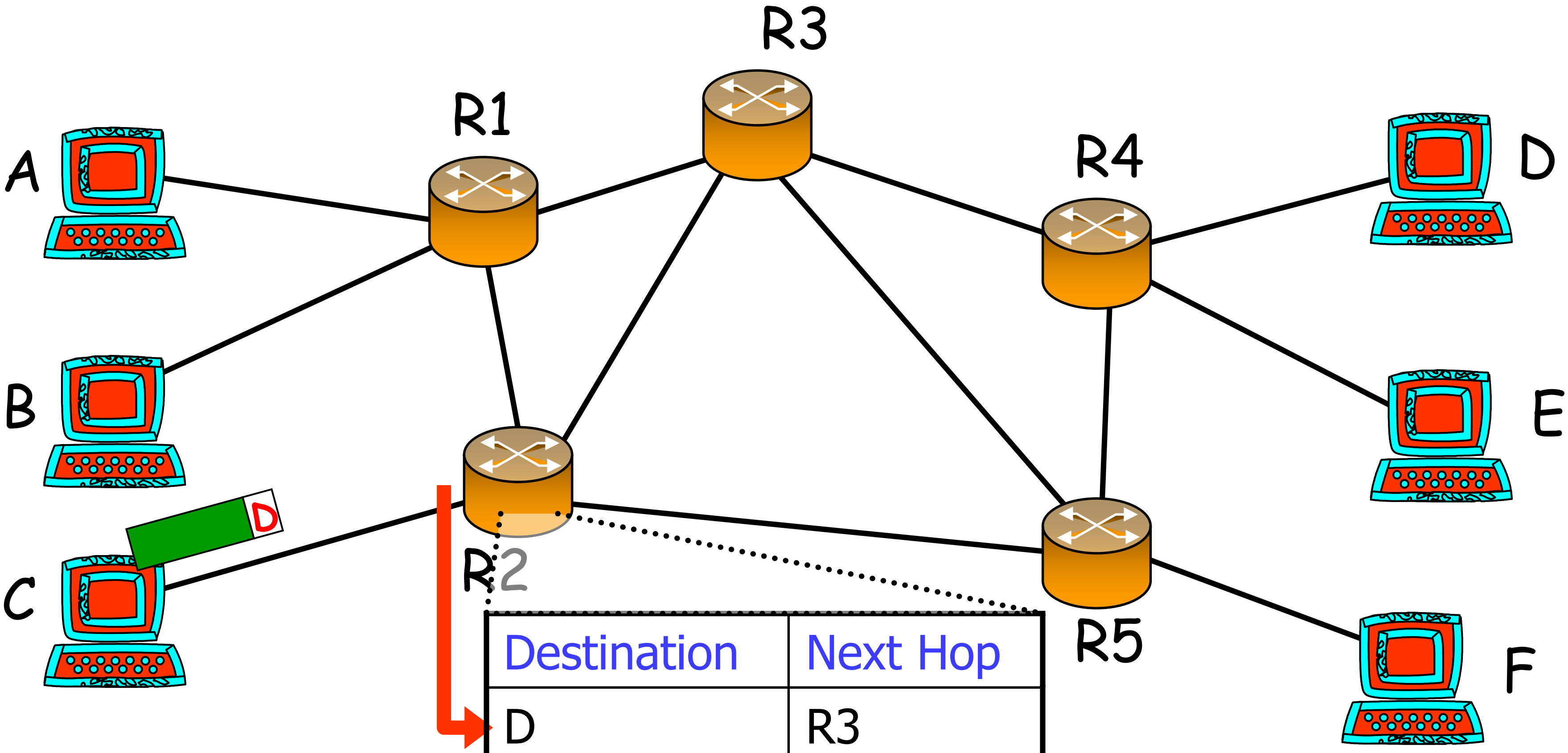
But what about this network…!?

The public Internet in 1999



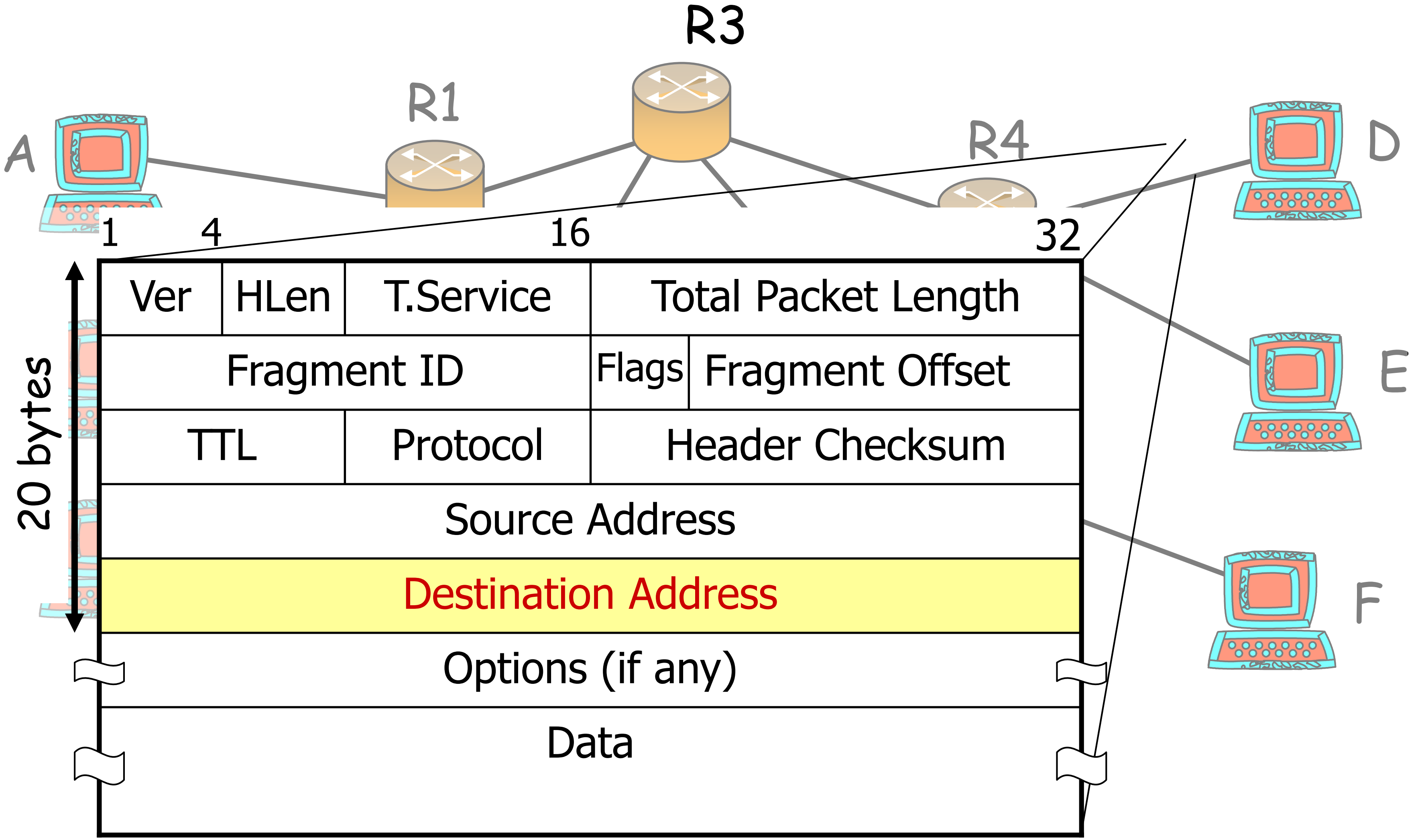
Learn more at
http://www.lumeta.com

What is Routing?


R3
R1
R4
A
D
B
E
D
C
R2
R5
F

Destination	Next Hop
D	R3
E	R3
F	R5

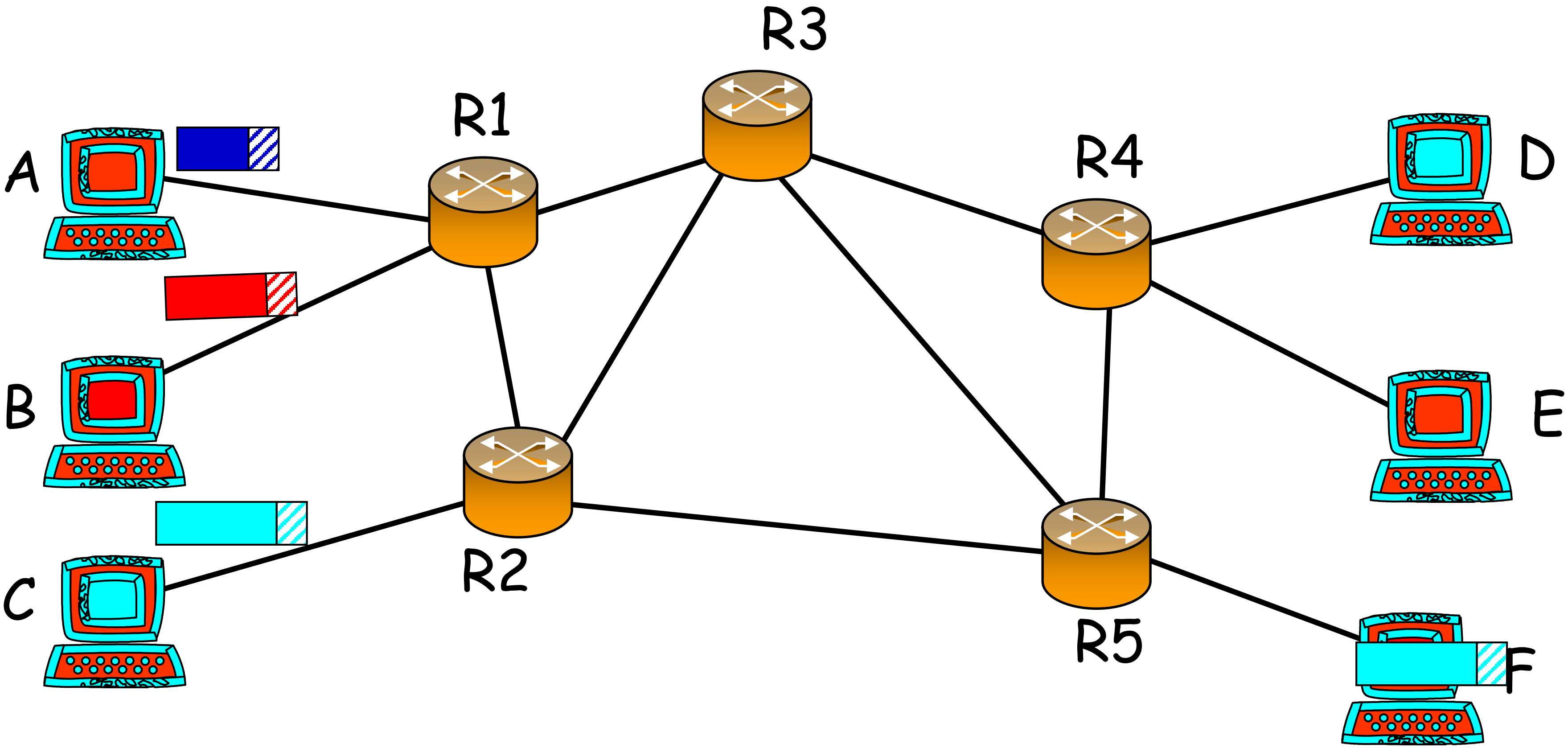
What is Routing?



1 4 16 32

<table>
<tr><td>Ver</td><td>HLen</td><td>T.Service</td><td colspan="2">Total Packet Length</td></tr>
<tr><td colspan="3">Fragment ID</td><td>Flags</td><td>Fragment Offset</td></tr>
<tr><td colspan="2">TTL</td><td>Protocol</td><td colspan="2">Header Checksum</td></tr>
<tr><td colspan="5">Source Address</td></tr>
<tr><td colspan="5">Destination Address</td></tr>
<tr><td colspan="5">Options (if any)</td></tr>
<tr><td colspan="5">Data</td></tr>
</table>

What is Routing?


R3
R1
R4
A
D
B
E
R2
C
R5
F

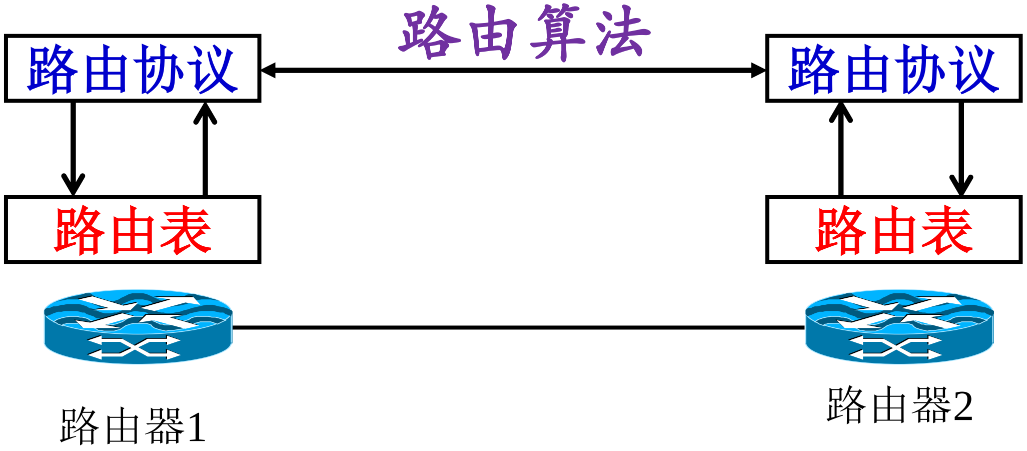
转发(Forwarding)与路由(Routing)

- IP分组**转发**包括两部分工作：
  1.怎样**将报文从路由器的输入端口送到输出端口**(“**IP 转发**”)？
  2. 怎样**建立和查找路由** (“**IP 路由**”)?
- **IP转发**由**IP协议**依据**路由表**完成
- **IP路由**由**路由算法**和**路由协议**完成

通过交换路由信息来动态更新维护路由表

路由算法的具体实现

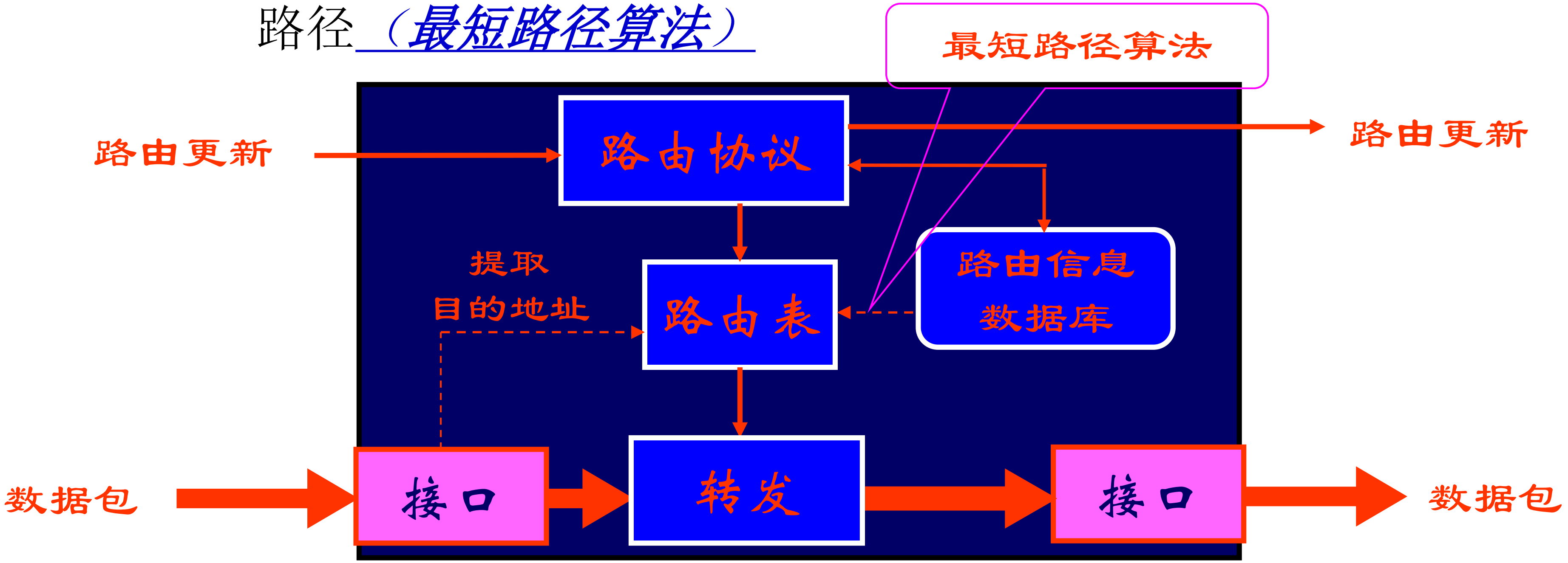
路由的主要内容


路由算法
路由协议
路由表
路由器1
路由协议
路由表
路由器2

- ✓**路由表**
- ✓**路由算法**（交换路由信息，更新路由表）
- ✓**路由协议**（路由算法的具体实现）

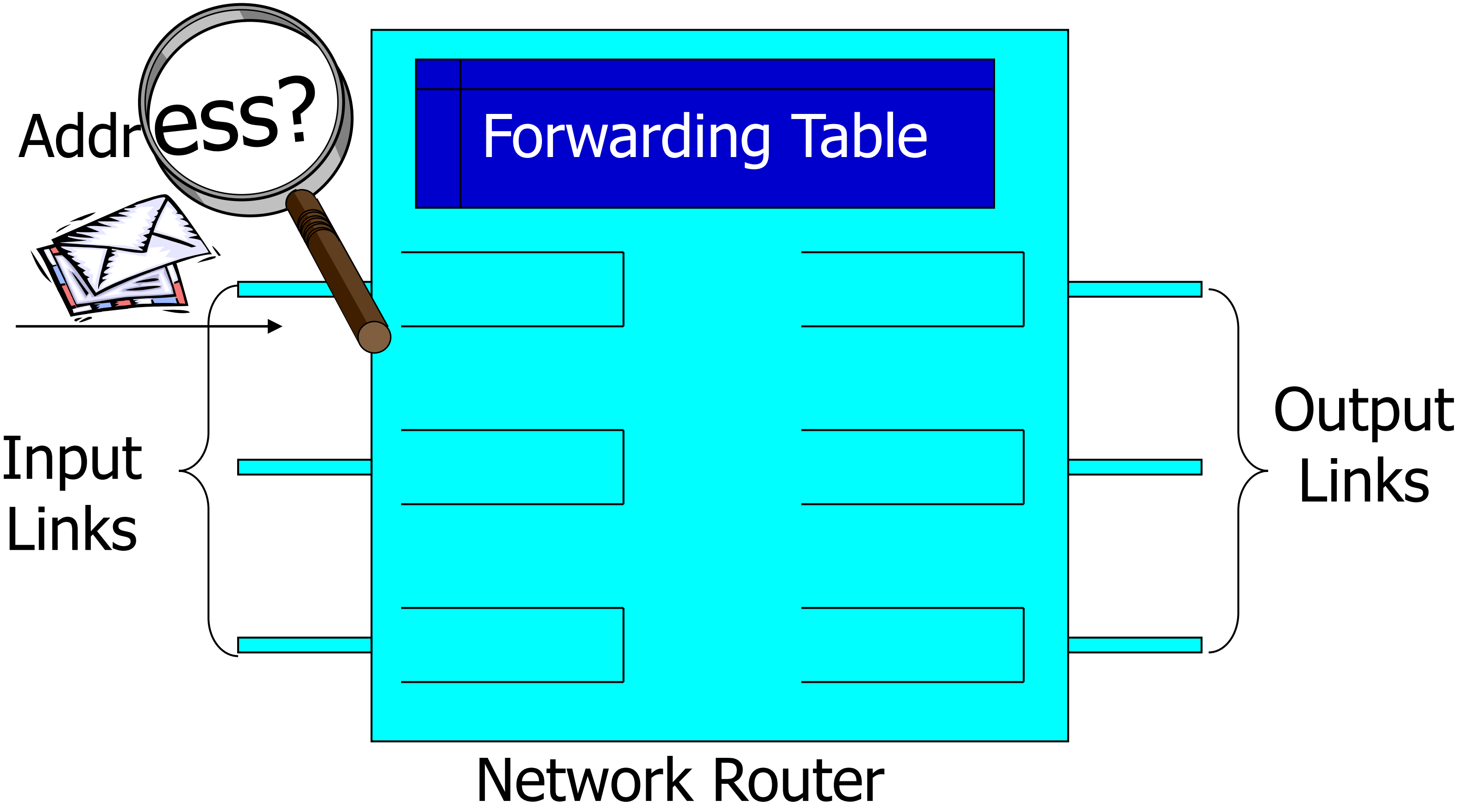
路由器

- 路由器的功能
  - 利用“最佳路径”将分组从一个网络转发到另一个网络 **(IP Forwarding)**
  - 与其它路由器交换路由信息（例如各网络的位置、可达性等），更新维护路由表----(***路由算法)***
  - 路由算法的实现----***（路由协议）***
  - 了解了网络拓扑结构信息以后，路由一般采用最佳路径***（最短路径算法）***


最短路径算法
路由更新
路由协议
路由更新
提取
目的地址
路由表
路由信息
数据库
数据包
接口
转发
接口
数据包

分组转发

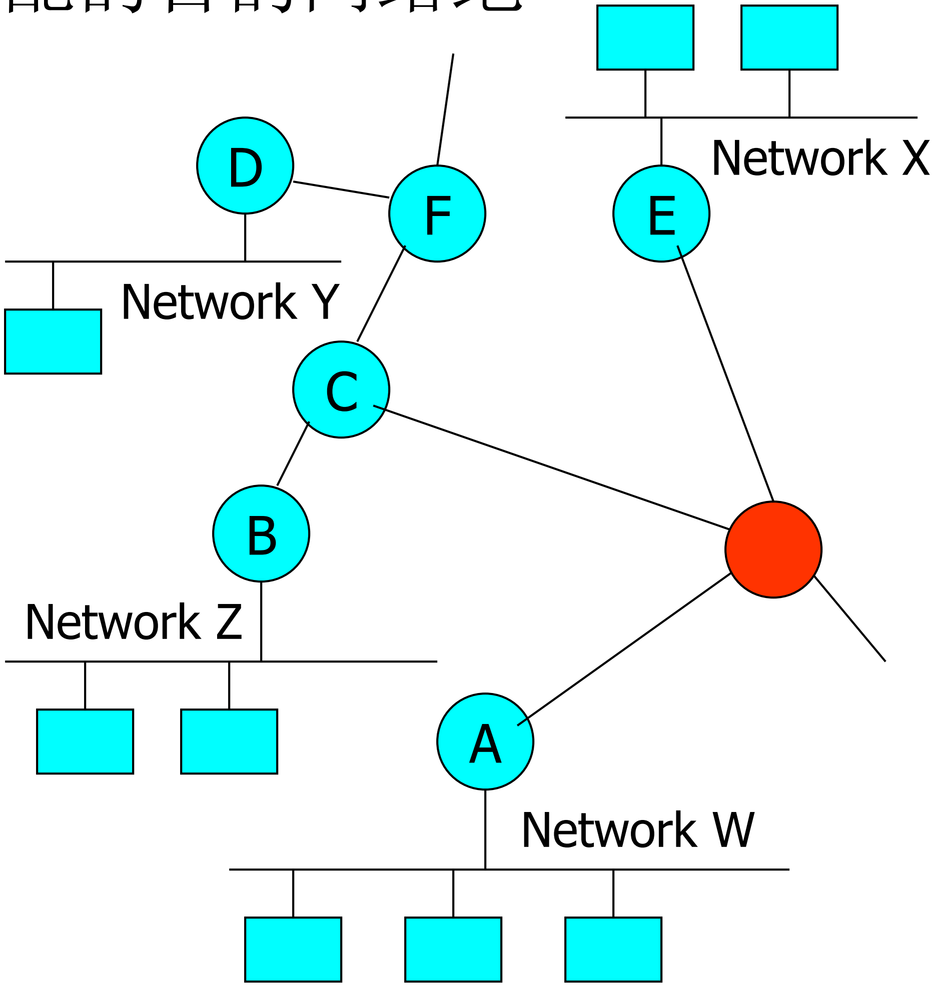
- **选路与转发**
- 网络中的源主机所发出的 IP 分组经过一个或多个路由器传输到另一个网络中的目的主机的过程


Address?
Forwarding Table
Input Links
Output Links
Network Router

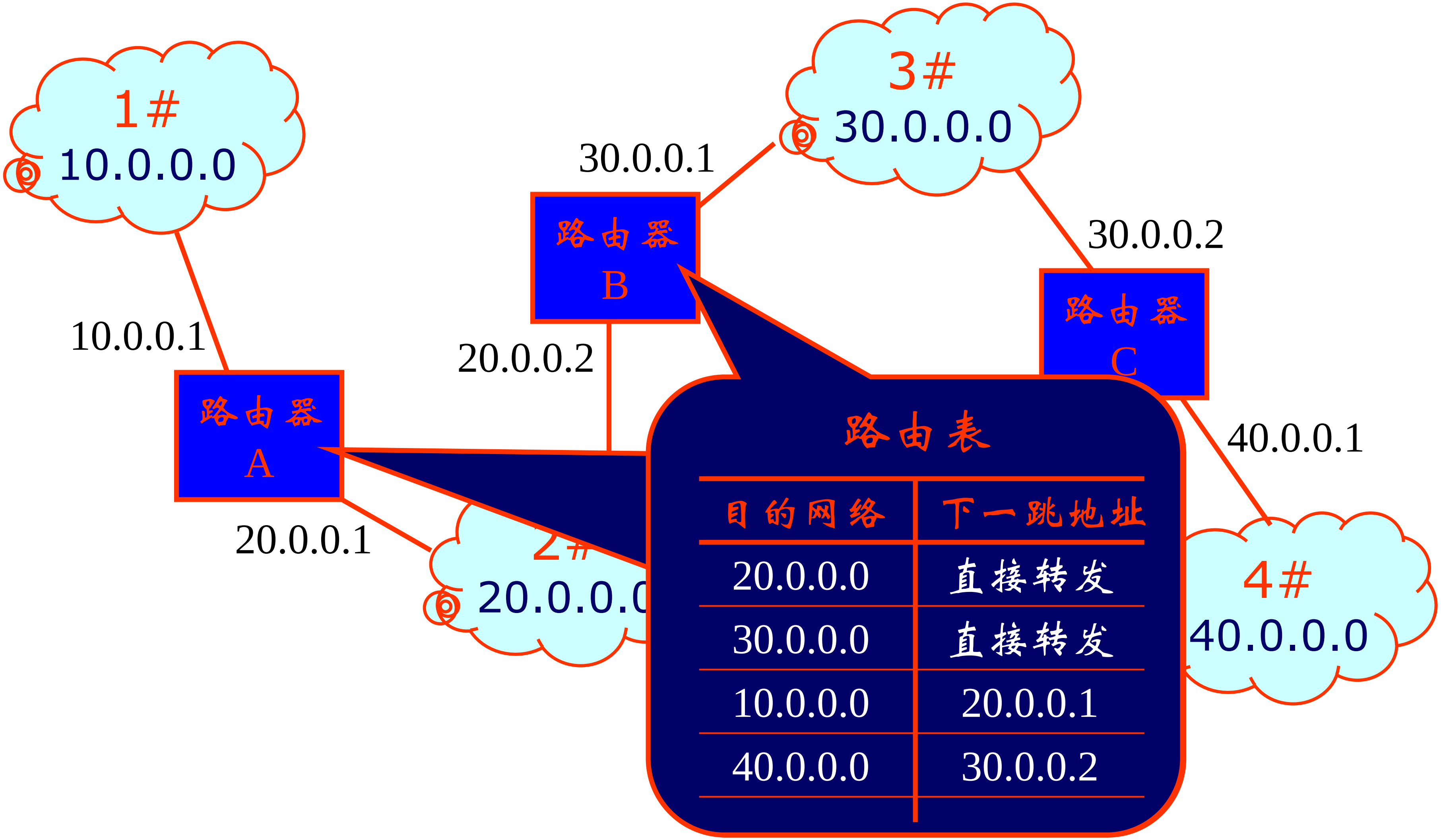
路由表

- ☐ 基于网络地址
- ☐ 路由器找一个匹配的目的网络地址来决定下一跳

Destination	Next hop (port)
Network X	E
Network Y	C
Network Z	C
Network W	A


D
F
E
Network X
Network Y
C
B
Network Z
A
Network W

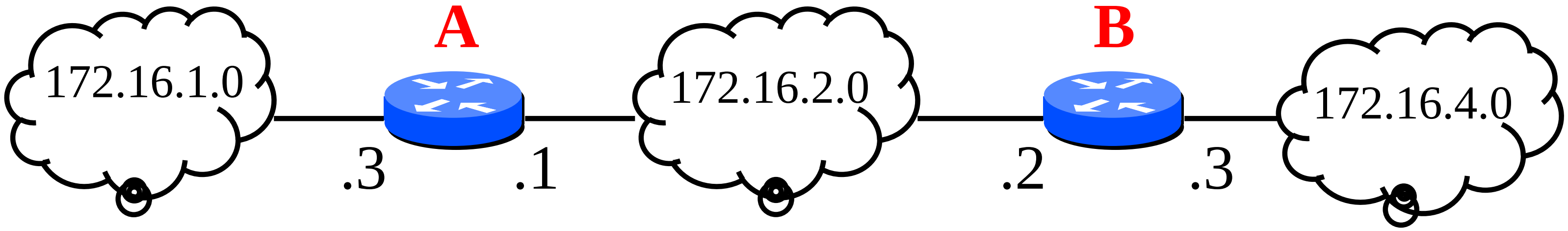
一个路由表实例


1#
10.0.0.0
3#
30.0.0.0
30.0.0.1
路由器
B
30.0.0.2
路由器
C
10.0.0.1
20.0.0.2
路由器
A
40.0.0.1
20.0.0.1
2#
20.0.0.
4#
40.0.0.0

路由表

目的网络	下一跳地址
20.0.0.0	直接转发
30.0.0.0	直接转发
10.0.0.0	20.0.0.1
40.0.0.0	30.0.0.2

路由表基本结构


A
B
172.16.1.0
172.16.2.0
172.16.4.0
.3
.1
.2
.3

路由器**A**的路由表

目的网络地址	路径 （下一跳结点）	路径成本 （距离跳数）
172.16.1.0	直接转发	0
172.16.2.0	直接转发	0
172.16.4.0	172.16.2.2	1

❑路径：用下一路由器（下一跳）对应的网络接口的**IP**地址来表示。

❑路径成本/开销：用路径的距离、跨越的路由器的个数（跳数 **hop**）、时延等度量。

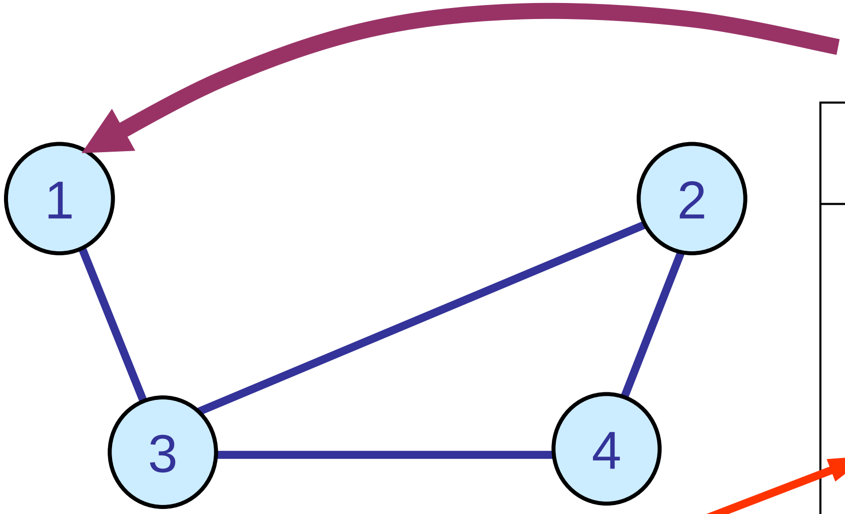
路由表中的路由类型

- **普通路由**：路由表的主要表项都是基于**网络地址**的
  - **目的**：减小路由表的长度，提高路由算法的效率
  - **类比**：邮政局在城市间转发信件依据的是城市的邮政编码
- **默认路由**：路由表没有明确指明一条到达目的网络的路由信息时，就将分组转发到默认路由指定的路径

  (**目的**：默认路由用来减少路由表的规模)
- **特定主机路由**（host-specific routing）：目的地址为特定网络中的特定主机，对特定的某个主机（而不是网络）指定一条特别的路径，即：目的IP地址/32

在路由表中使用默认路由

以结点 1 的路由表为例来讨论



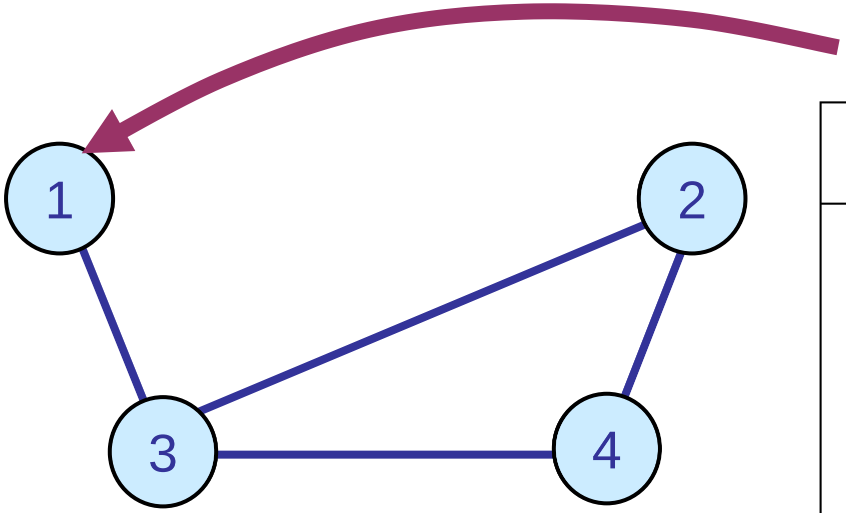
结点 1 的转发表

目的站	下一跳
1	直接
2	3
3	3
4	3

可以合并

这三个项目的“下一跳”都是转发到“3”（结点 3）。

在路由表中使用默认路由



结点 1 的转发表

目的站	下一跳
1	直接
默认	3

默认路由

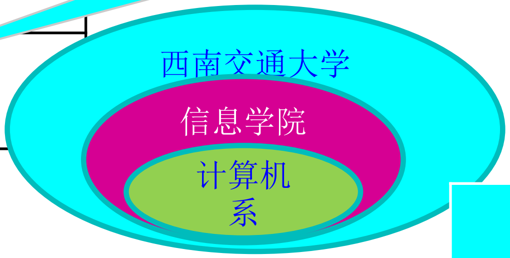
普通路由、主机路由和默认路由实例

目的地址	下一跳
142.150.64.0 / 24	A
142.150.71.128 / 28	**B**
142.150.64.0 / 30	C
142.150.0.0 / 16	D
142.150.71.132 / 32	A
0. 0. 0. 0/0	E

普通路由：
路由到网络
(指定目的网络号)

特定主机路由
(指定主机号)

默认路由
(用全0地址表示)



选路由的时候需要进行**最长网络地址匹配**

142.150.71.128 / 28 & 142.150.71.132 / 32

去往	网络号	子网掩码
西南交通大学	XXX	
信息学院	YYY	
计算机系统	ZZZ	

6.3.2 路由策略

- 固定路由
- 泛洪法
- 随机路由
- 自适应路由

固定路由

依据网络拓扑结构图生成路由表

节点1的路由表

目的站	下一站
1	-
2	3
3	3
4	3

节点2的路由表

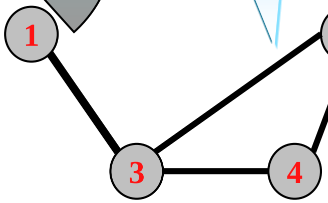
目的站	下一站
1	3
2	-
3	3
4	4

节点3的路由表

目的站	下一站
1	1
2	2
3	-
4	4

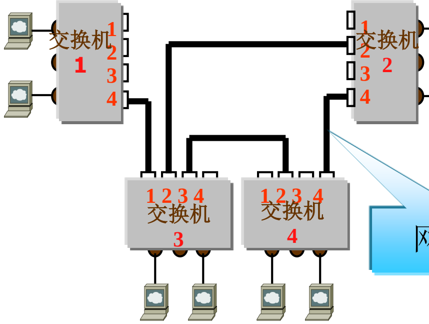
节点4的路由表

目的站	下一站
1	3
2	2
3	3
4	-



网络的拓扑结构图

Two


交换机 1
1
2
3
4
交换机 2
1
2
3
4
1 2 3 4
交换机 3
1 2 3 4
交换机 4

网络结构

One

Three

节点1的路由表

目的站	下一站
1	-
*	3

节点2的路由表

目的站	下一站
2	-
4	4
*	3

节点3的路由表

目的站	下一站
1	1
2	2
3	-
4	4

节点4的路由表

目的站	下一站
2	2
4	-
*	3

*表示具有相同"下一站"的项目-默认路由

使用默认路由的简化路由表

多选择路由

如何依据路由表完成选路？

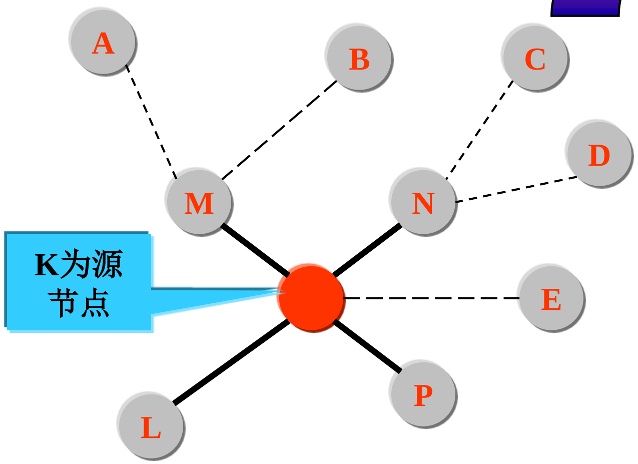


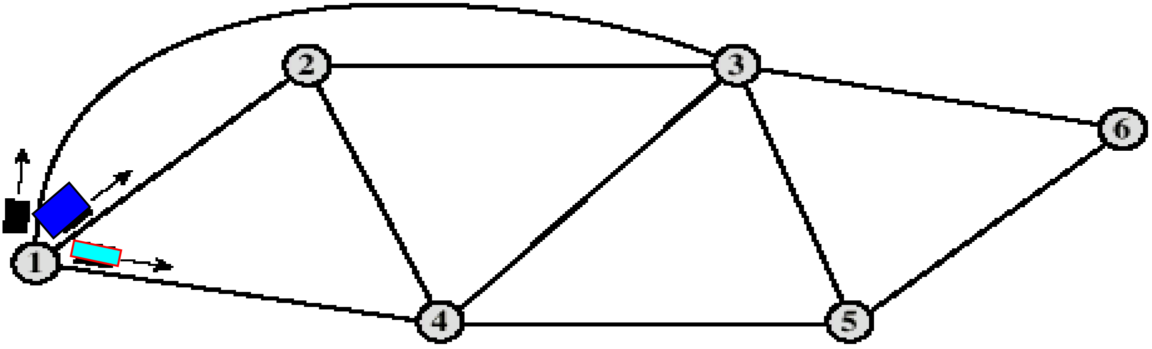
图 网络拓扑示意图，实线表示节点间有直接链路，虚线表示节点间无直接链路

目的站	下一站	概率	下一站	概率	下一站	概率
U	M	0.50	L	0.40	N	0.10
V	M	0.35	N	0.35	L	0.30
W	N	0.65	M	0.25	P	0.10
X	N	0.55	P	0.25	M	0.15
Y	P	0.45	N	0.30	M	0.25
....	....	....	....	....	....	....

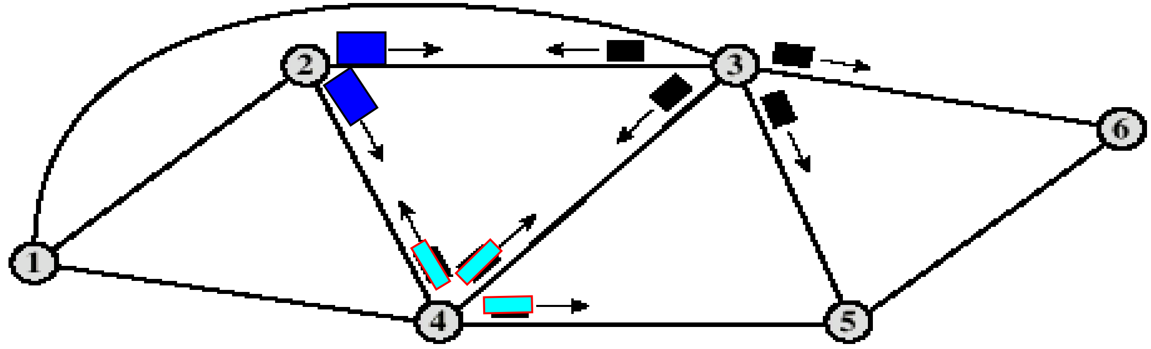
图 节点K的路由表

泛洪法

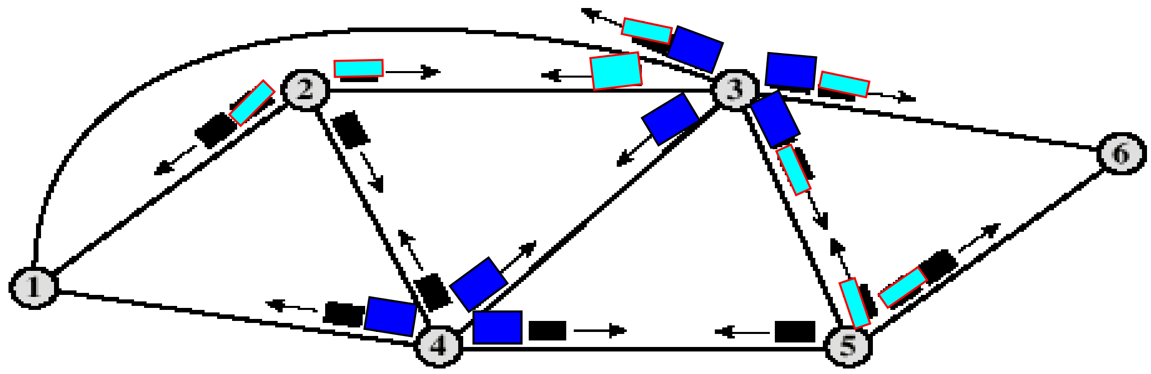
节点收到报文后**向除了报文进入端口外的其它所有端口转发**



(a) First hop



(b) Second hop



(c) Third hop

泛洪法的优点

- **尝试所有的路由**
  - 很健壮
- **至少一个报文以最短路径到达**
  - 可以用来建立**虚电路**;
  - 作为路由算法的性能**评价基准**
- **访问所有节点**
  - 对分发信息很有用 (e.g. 用于分发信息)

随机路由

- 节点为到来的报文**随机地**选择一条传输路径
- 对于多条出线，基于概率计算来随机选择一条输出线
- **优点**：不需要了解关于网络结构信息
- **缺点**：所选中的路由通常既不是最低代价也不是最小跳段

自适应路由

- 大部分的分组交换网络均使用自适应路由算法
- **自适应**：路由随网络状态的变化而改变
  - 失效(Failure)
  - 阻塞
- 需要了解网络结构信息
- 路由计算更为复杂
- 通常需要在路由质量和开销之间进行折衷：
- 自适应过于敏捷会导致振动
- 反应慢一点比较恰当

自适应路由算法的分类

- 依据**自适应的信息来源**，自适应路由算法分为：
  - **本地节点**(孤立的)
    - 路由到最短队列里的输出线（**热土豆法**）
    - 没有充分利用容易获得的路由信息
  - **邻接节点**（DV路由算法）
  - **所有节点**（LS路由算法）

动态路由算法

- Internet中的两种基本路由算法.

1. 距离矢量路由算法 (DV)

- 每个节点通过测试都知道到与它相邻的节点的距离
- 节点周期性地把路由表中的VD信息发送给它的**邻接点**
- 邻接节点接收到VD路由信息后更新自己的路由表。通过多次更新路由表最终收敛
- 新节点通知它们的邻居

2. 链路状态路由算法 (LS)

- 每个节点知道到自己邻接点的距离
- 定期或状态改变时将链路状态信息发送给网络中的**所有节点**
- 这样，每个节点都了解网络的结构(链路状态)
- 每个节点独立地采用**最短路径算法**生成路由表

Internet中的路由算法及其实现协议

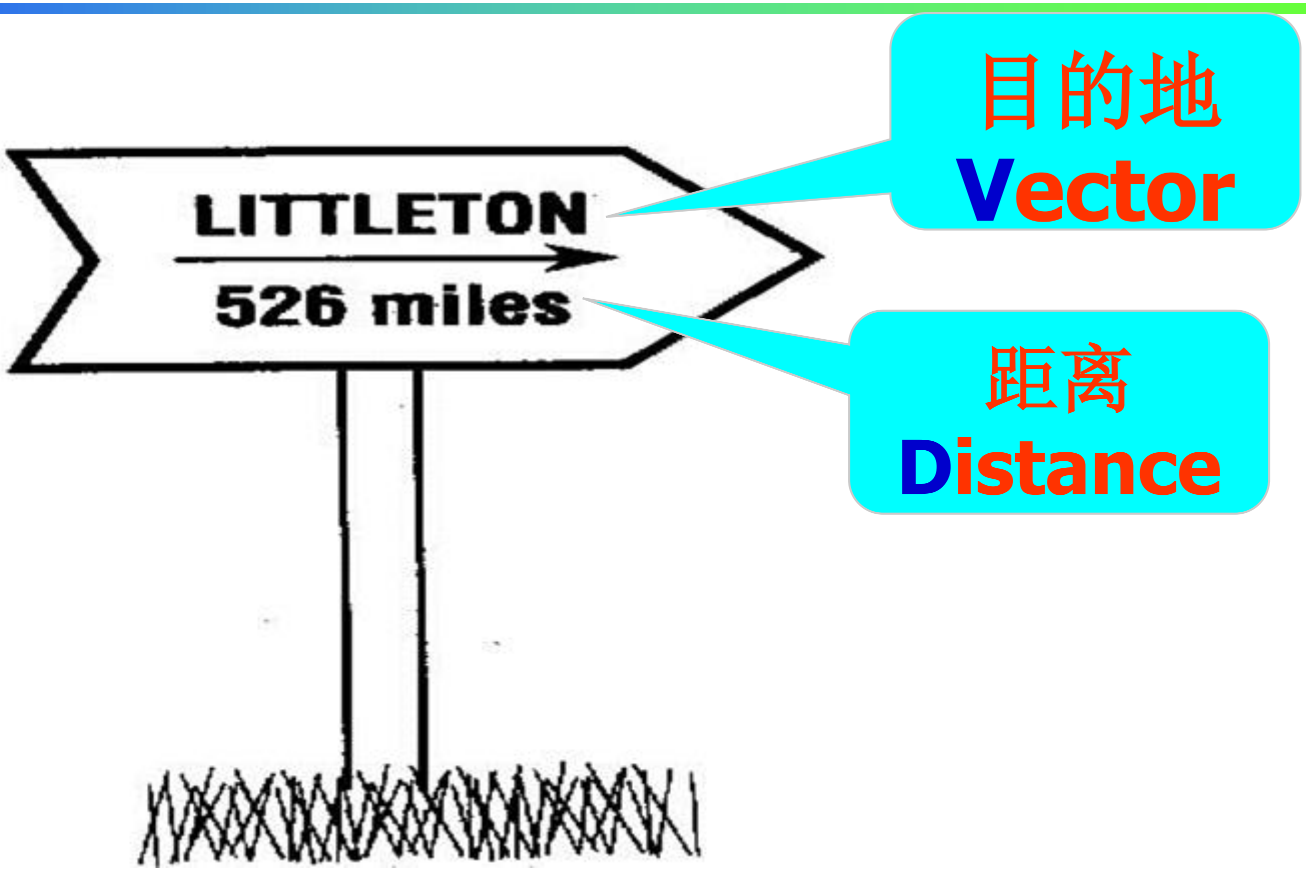
距离矢量(DV)

- **路由信息协议 (RIP)**
- 网关到网关协议 (GGP)
- 外部网关协议 (EGP)
- 内部网关路由协议 (IGRP)

链路状态(LS)

- 开放最短路径优先 **(OSPF)**
- 中介系统 –中介系统 (IS-IS)

距离矢量（DV）路由算法


LITTLETON
526 miles

目的地
Vector

距离
Distance

DV = 设立路标(目的地V，距离D)，每个目的地一个

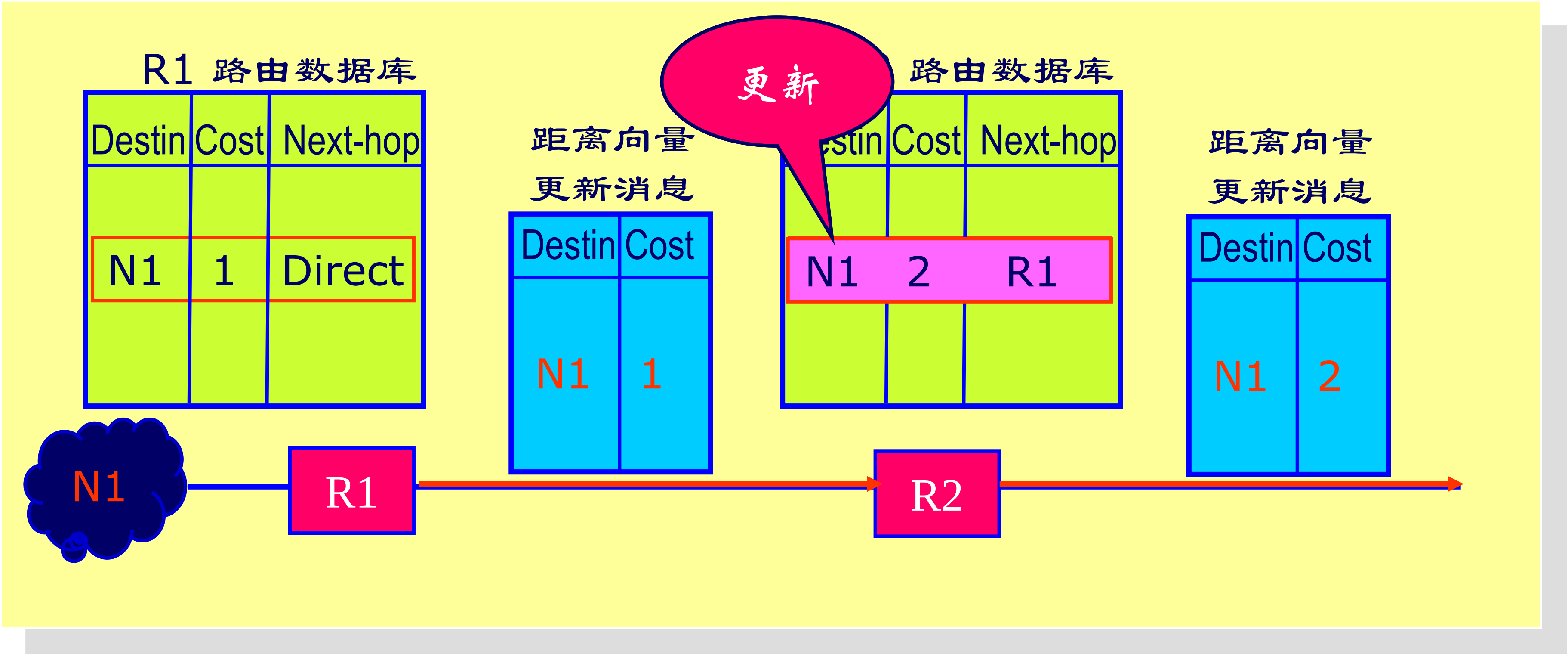
距离矢量路由算法基本思想

- ➢ 基本思想

- ➢ 距离向量：
- ➢ 路由表的构成
- ➢ 距离向量的更新过程

各路由器都将其所了解的路由信息（已知的可到达的所有网络以及到达各网络的距离等）发送给相邻的路由器。

- □ 即一个二元组{network，cost}
- □ network：代表目的网络
- □ cost：去往该网络的距离或费用


R1 路由数据库
Destin Cost Next-hop
N1 1 Direct
距离向量
更新消息
Destin Cost
N1 1
更新
R2 路由数据库
Destin Cost Next-hop
N1 2 R1
距离向量
更新消息
Destin Cost
N1 2
N1
R1
R2

路由刷新操作

刷新时，逐项检查来自其相邻路由器的VD信息，遇到下列情况之一时，修改本地路由表（假定路由器Gi收到其相邻路由器Gj的VD报文）：

（A）**新结点进网**（**由不可达变成可达**）。Gj列出的某目的网络在Gi的路由表中没有，则在Gi路由表中添加相应的表目，其“目的地”是Gj表目中的目的地，“距离”为Gj表目中的距离值加1，其“下一跳”为Gj;

（B）**路径更改**(存在另一条经由Gj的**更短路径**，经由Gn改为经由Gj)。Gj去往某信宿的距离值比Gi去往该信宿的距离值减1还小，这表明Gi去往该信宿若经由Gj距离会更短，这时应改选更短的路径，其“目的地”不变，“距离值”为Gj表目中的距离值加1，“下一跳”为Gj。

路由刷新操作（续）

（3）**所有经由Gj的路由均需要随Gj变化**。若Gi去往某信宿的路径经由Gj，而Gj去往该信宿的路径发生了变化，则Gi的路由需要随之改变。有两种情况：

（A）（**由可达变成不可达**）Gj的VD选径表中不再包含去往某目的地的路径(不可达)，则Gi路由表中相应的表目必须删除；

（B）（经由Gj的路径**变优或变劣**，Gi随之变优或变劣）G_j的VD表中去往某目的地的路径的距离发生了变化，则G_i中相应的表目的“距离值”必须随之修改，以G_j中的距离值加1来取代原距离值。

距离向量算法

路由器收到相邻路由器（其地址为 X）的VD向量：

(1) 先修改VD向量中的所有项目：把“下一跳”字段中的地址都设为 X，并把所有的“距离”字段的值加 1。

(2) 对修改后的 VD向量中的每一个项目，重复以下步骤：

- 若项目中的目的网络不在路由表中，则把该项目添加到路由表中（新站点入网）。
  - 否则
    - 若下一跳字段给出的路由器地址是同样的，则用收到的项目替换原路由表中的项目。
      - 否则
        - 若收到项目中的距离小于路由表中的距离，则进行路由更改（更短路由），
          - 否则，什么也不做。

(3) 若 3 分钟还没有收到相邻路由器的VD向量，则把此相邻路由器记为不可达路由器，即将距离置为 16（表示不可达）。

(4) 返回。

例1. 根据VD修改

Gj发送给Gi的VD信息

信宿	距离（跳）
10.0.0.0	1
30.0.0.0	4
40.0.0.0	3
41.0.0.0	2
180.0.0.0	5

更短路径

新节点进网

假定Gi和Gj为相邻路由器

Gi的VD信息

信　宿	距离（跳）	路径
10.0.0.0	0	直接
30.0.0.0	7	G_n
40.0.0.0	3	G_j
45.0.0.0	4	G_k
180.0.0.0	7	G_j
190.0.0.0	10	G_m
199.0.0.0	6	G_j

变劣

变优

不可达

路由器Gi更新前的路由表

信宿	距离（跳）	路径
10.0.0.0	0	直接
30.0.0.0	5	Gj
40.0.0.0	4	Gj
41.0.0.0	3	Gj
45.0.0.0	4	Gk
180.0.0.0	6	Gj
190.0.0.0	10	Gm

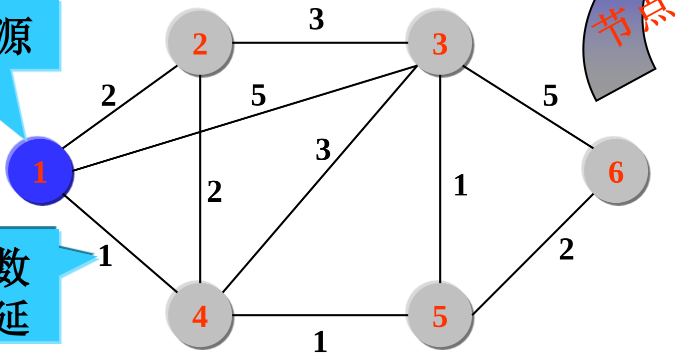
新路径

Gi更新后的路由表

例2. 以时延为度量的DV算法例子

定期向相邻路由器通报

节点1为信源



链路上的数字表示时延

图1 网络拓扑示意图

节点1的路由表

目的节点	时延	下一站节点
1	0	-
2	2	2
3	5	3
4	1	4
5	6	3
6	8	3
	D_1	S_1

节点1到节点3,5,6时延并不是最小时延，需要根据相邻路由器的VD路由信息更新.

图2 节点1更新前的路由表

更新

节点1收到相邻节点通报的DV向量

D_2	D_3	D_4
2	3	1
0	3	2
3	0	2
2	2	0
3	1	1
5	3	3

在交换路由信息过程中节点1收到节点2,3,4的时延向量后，更新路由表(每收到一个DV信息就更新一次)

目的节点	时延	下一站节点
1	0	-
2	2	2
3	5	3
4	1	4
5	5	2
6	7	2

先依据D2更新

目的节点	时延	下一站节点
1	0	-
2	2	2
3	3	4
4	1	4
5	5	2
6	7	2

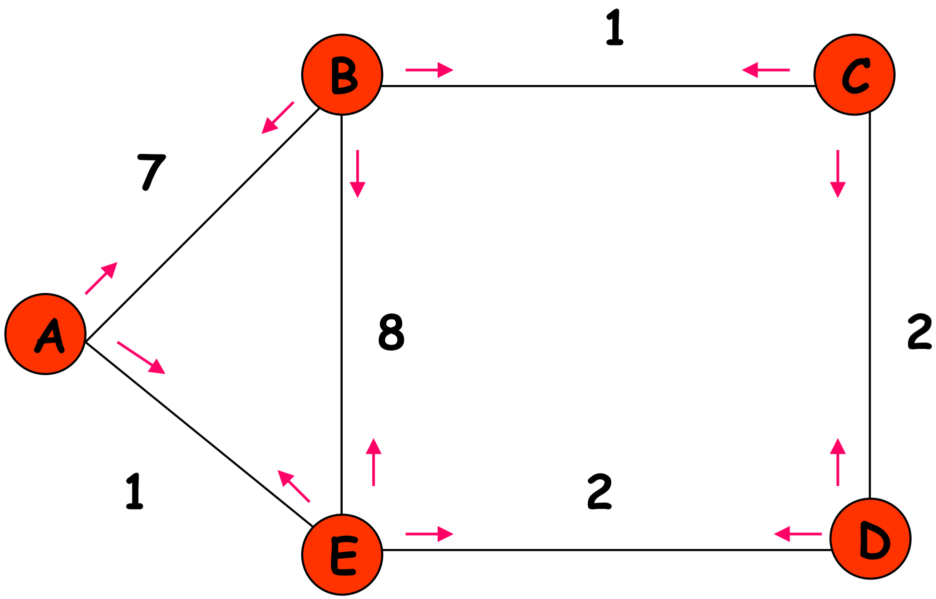
再依据D3更新

目的节点	时延	下一站节点
1	0	-
2	2	2
3	3	4
4	1	4
5	2	4
6	4	4

最后依据D4更新

图3 节点1收到相邻节点2,3,4的时延向量后对路由表进行更新

例3- 通过交换路由信息动态更新路由表实例--初始路由表

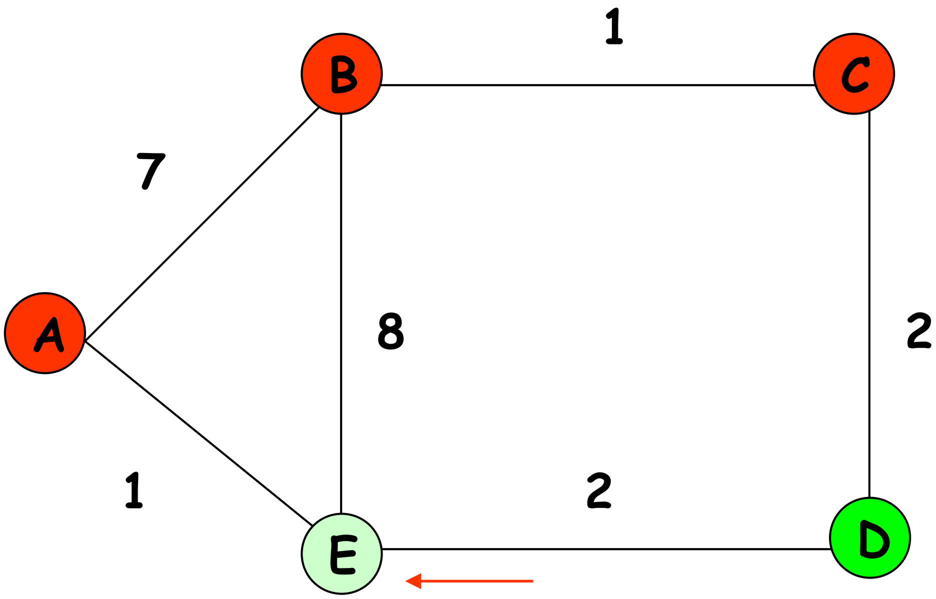


Info at node	Distance to node				
	A	B	C	D	E
A	0	7	~	~	1
B	7	0	1	~	8
C	~	1	0	2	~
D	~	~	2	0	2
E	1	8	~	2	0

初始化时，通过测试，所有节点都能知道其相邻节点及其距离，得到**初始路由表**。

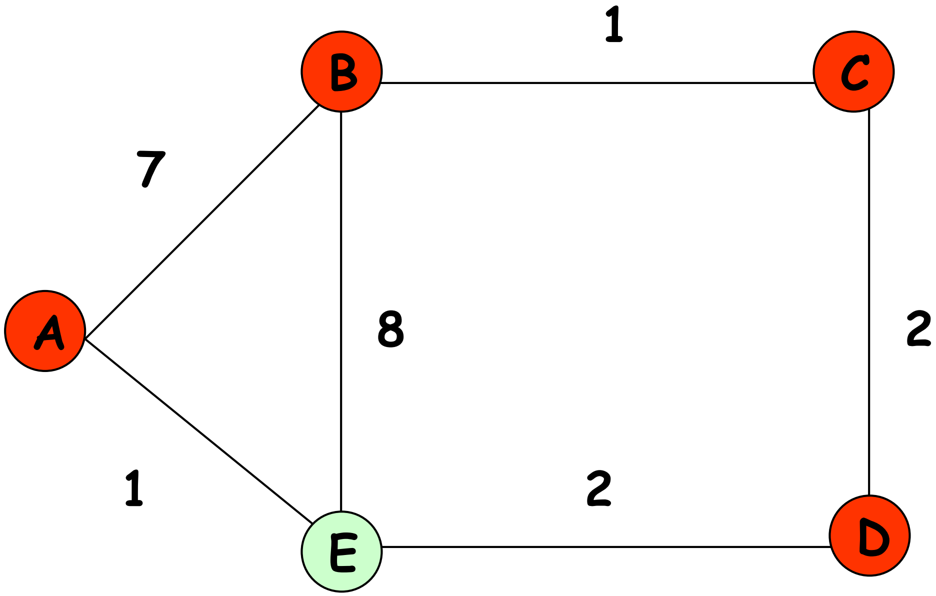
然后相邻节点之间**相互交换路由信息**来更新路由表。

E接收D的路由信息


1
B
C
7
8
2
A
1
2
E
D

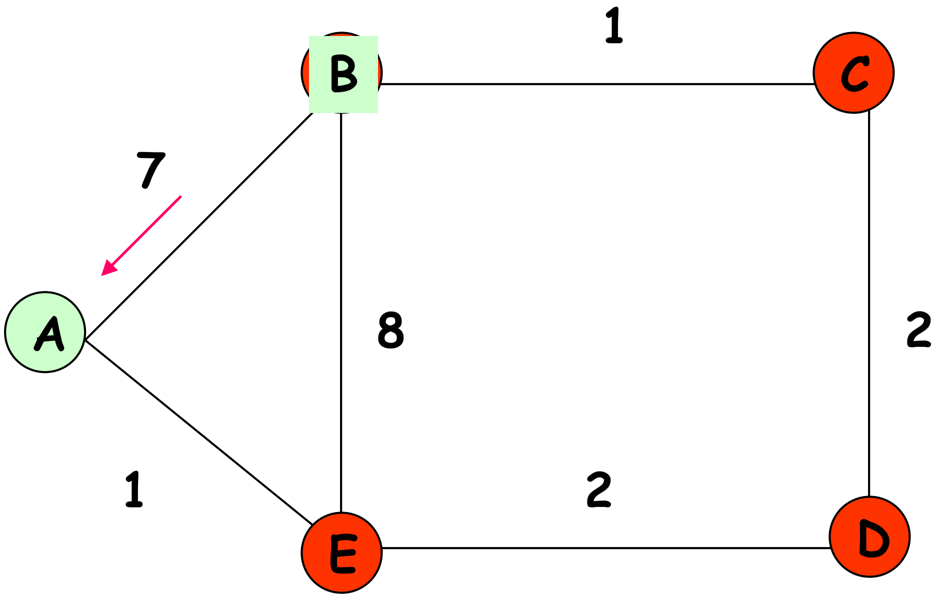
Info at node	Distance to node				
	A	B	C	D	E
A	0	7	~	~	1
B	7	0	1	~	8
C	~	1	0	2	~
D	~	~	2	0	2
E	1	8	~	2	0

E更新到C的代价


1
B
C
7
8
2
A
1
2
E
D

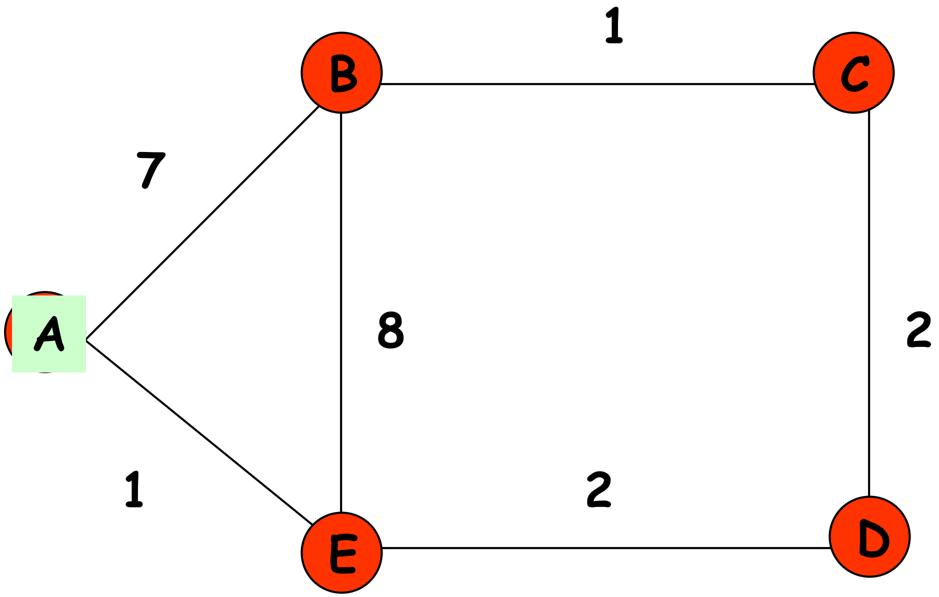
Info at node	Distance to node A	B	C	D	E
A	0	7	~	~	1
B	7	0	1	~	8
C	~	1	0	2	~
D	~	~	2	0	2
E	1	8	4	2	0

A接收B的路由信息


1
B
C
7
A
8
2
1
2
E
D

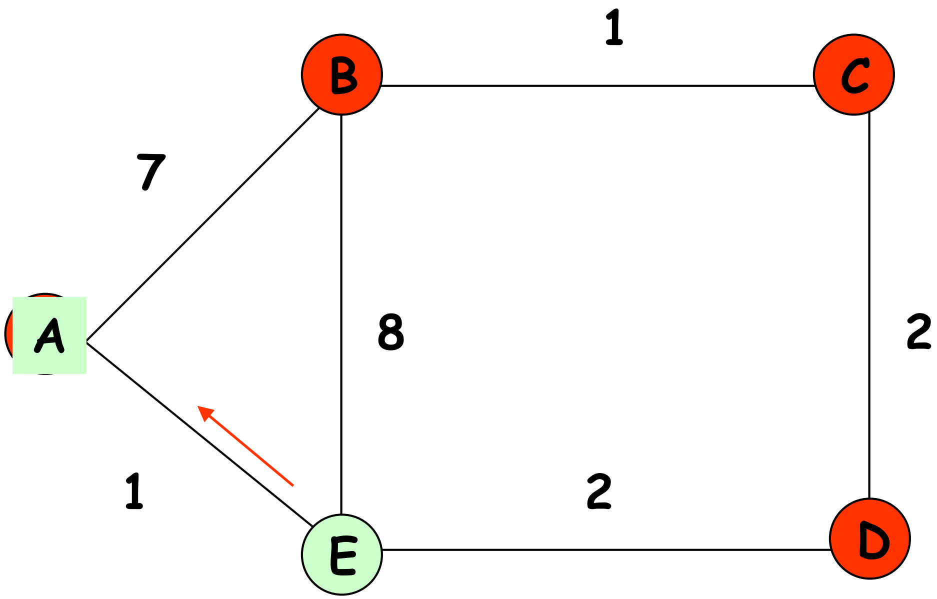
Info at node	Distance to node				
	A	B	C	D	E
A	0	7	~	~	1
B	7	0	1	~	8
C	~	1	0	2	~
D	~	~	2	0	2
E	1	8	4	2	0

A更新到C的代价


1
B
C
7
A
8
2
1
2
E
D

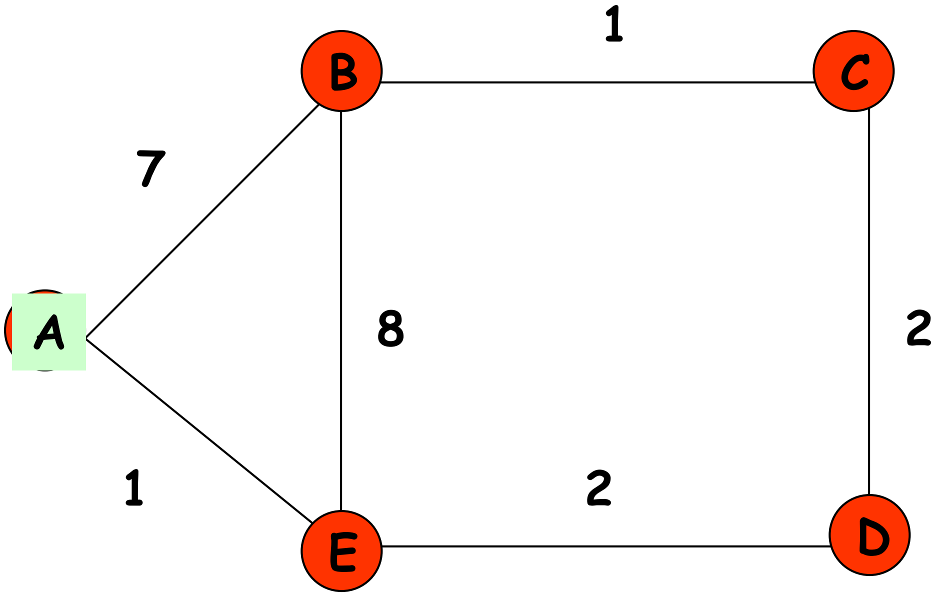
Info at node	Distance to node A	B	C	D	E
A	0	7	8	~	1
B	7	0	1	~	8
C	~	1	0	2	~
D	~	~	2	0	2
E	1	8	4	2	0

A接收E的路由信息


B
C
A
E
D
1
7
8
2
1
2

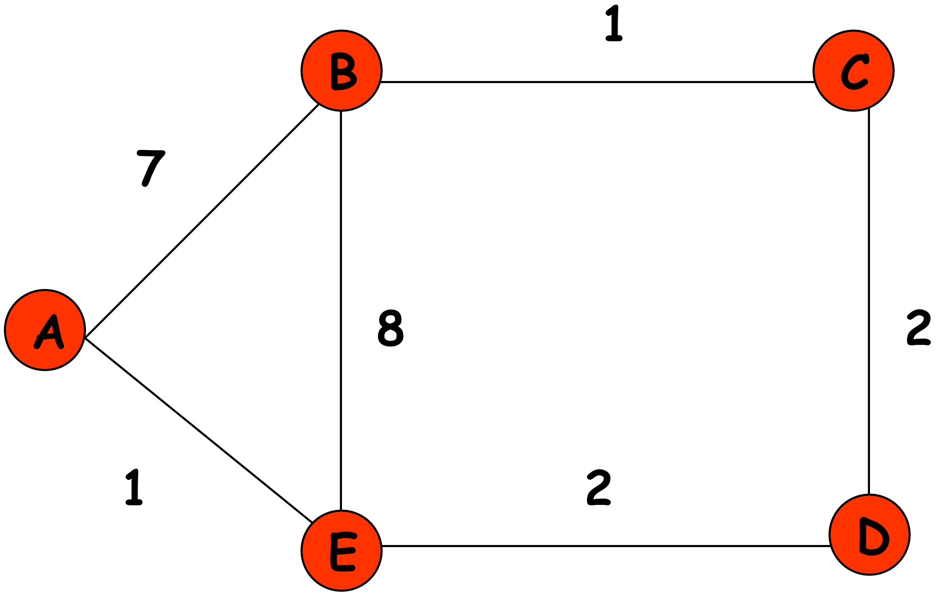
Info at node	Distance to node A	B	C	D	E
A	0	7	8	~	1
B	7	0	1	~	8
C	~	1	0	2	~
D	~	~	2	0	2
E	1	8	4	2	0

A更新到C和D的代价


A
B
C
D
E
7
1
1
8
2
2

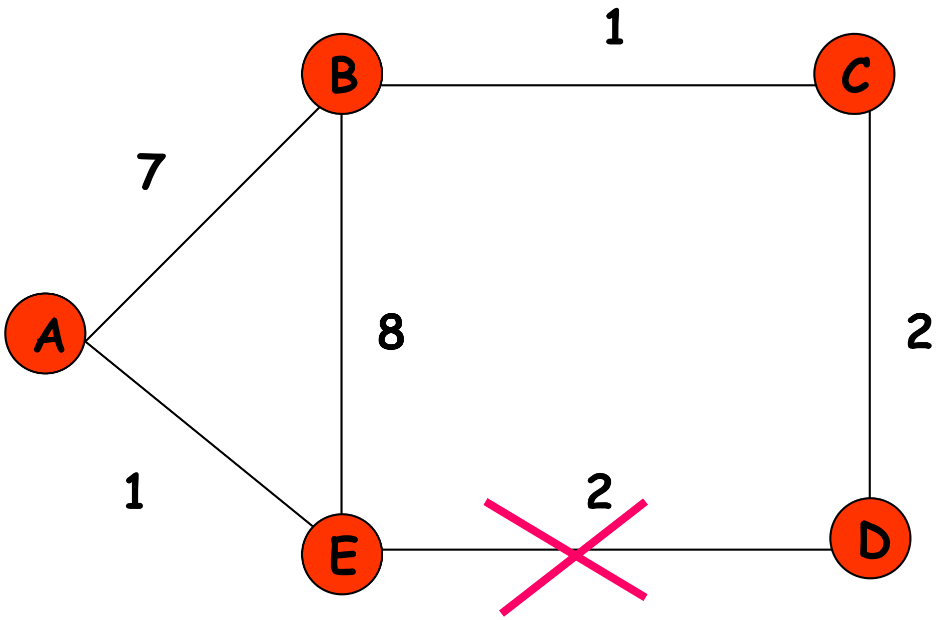
Info at node	Distance to node				
	A	B	C	D	E
A	0	7	5	3	1
B	7	0	1	~	8
C	~	1	0	2	~
D	~	~	2	0	2
E	1	8	4	2	0

最终收敛后的路由表


A
B
C
D
E
7
1
1
8
2
2

Info at node	Distance to node A	B	C	D	E
A	0	6	5	3	1
B	6	0	1	3	5
C	5	1	0	2	4
D	3	3	2	0	2
E	1	5	4	2	0

链路失败后更新路由


A
B
C
D
E
7
1
8
2
1
2

Info at node	Distance to node A	B	C	D	E
A	0	7	8	10	1
B	7	0	1	3	8
C	8	1	0	2	9
D	10	3	2	0	11
E	1	8	9	11	0

链路失败后的路由更新

DV算法的特点

V-D算法的**优点**：简单，易于实现。 **缺点**：

（1）不适合于路径变化剧烈或大型网间网环境，因为其路由器的路径变化象波动一样地从相邻路由器传播出去-----收敛慢（**无穷计数**），期间可能出现路径不一致；（**对好消息的反应迅速，但对坏消息却反应迟钝**）

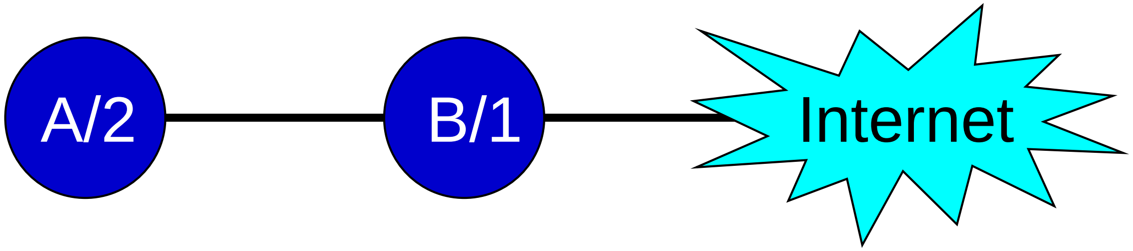
(2) 采用距离向量算法的路由协议(如RIP)规定距离最大值为15“跳”（16=∞，不可达），这对于一个大型网络来说是不够的，**只适用于中小型网络**。

（3）路由器之间交换完整的路由信息，需大量的信息交换----占用大量的网络带宽。

无穷计数（收敛慢）问题

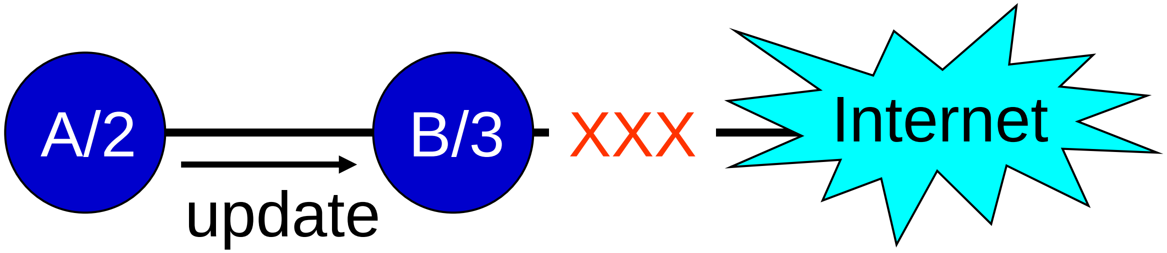
简单例子

- Costs in nodes are to reach Internet



- B到 Internet 的连接失败…，B测到后修改到Internet的路由为距离=16（不可达），即B/16
- 如果**B先向A发送路由信息** → A修改到 Internet 的路由为距离=16（不可达），马上更改，收敛
- 如果是**A先向B发送**路由信息，结果如何？

无穷计数（收敛慢）问题

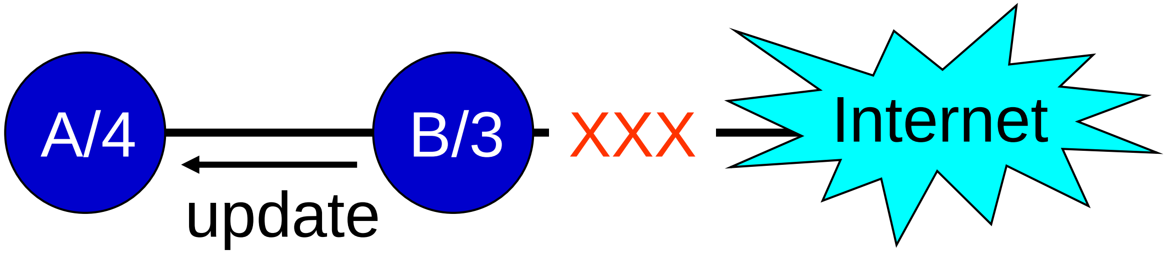

A/2
B/3
update
XXX
Internet

- B 收到A的路由信息（A/2）后，以为有一条从A到Internet，代价为2的路径

（注意：A并不知道该路由信息是B给定的！）

- 因此 B转到这条“更好”的路径，即将B的路由表改为B/3 (but wrong!)

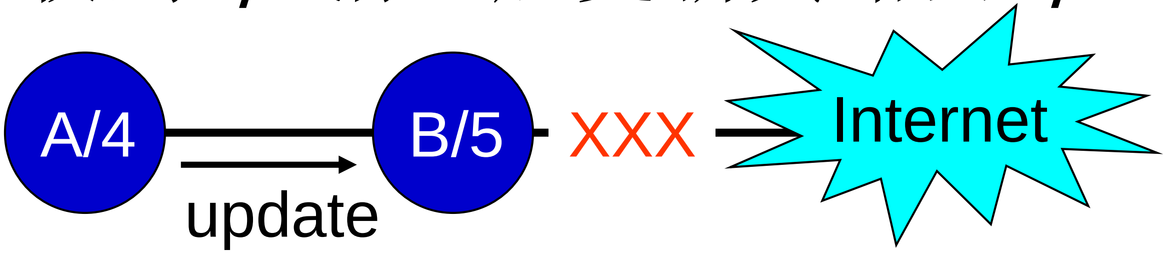
无穷计数（收敛慢）问题


A/4
B/3
XXX
Internet
update

- B向A发送路由信息B/3，A 收到后更新路由A/4
- 然后，A向B发送路由信息（A/4）

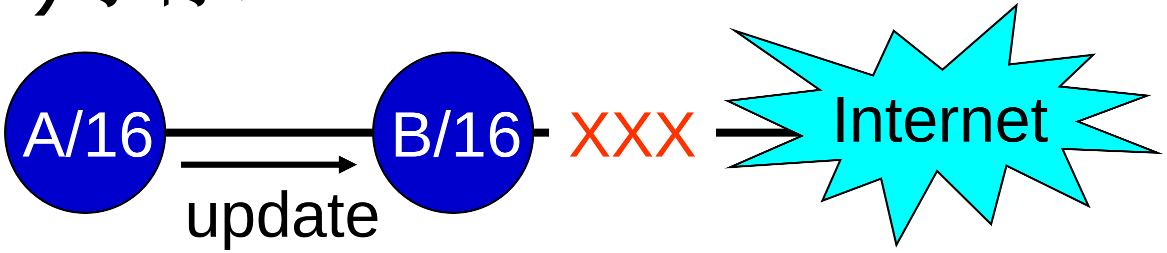
无穷计数（收敛慢）问题

- B 收到A/4消息后更新其路由B/5

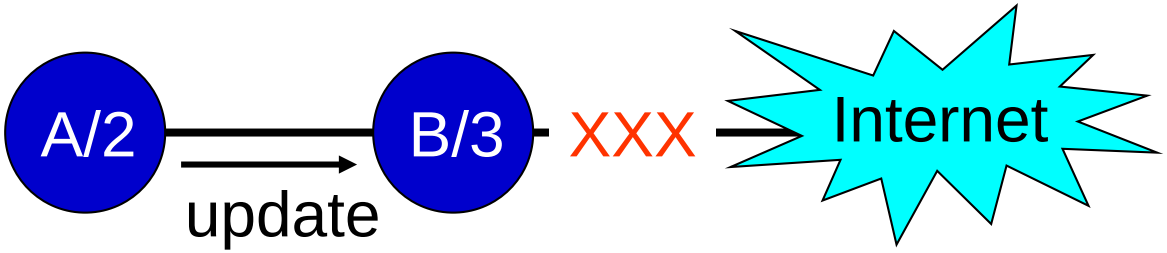

A/4
B/5
update
XXX
Internet

- ……

- 继续循环（7 个来回)，最终记数到无穷(16)才停止


A/16
B/16
update
XXX
Internet

造成慢收敛的原因


A/2
B/3
update
XXX
Internet

① B了解到不能到达Internet后**没有及时通报**

② B了解到**不能到达**Internet后**很快又接受**其它站点的有关Internet的**可达信息**

③ A发送给B的路由信息中**包含有经由B的路由**信息

避免无穷计数

- 保持法（Hold Down）
    - 在得知某个目的网络不可到达的一段时间内（RIP规定为60秒（2个路由信息交换周期）），路由器不接受关于该目的网络的任何可达的信息。
- 水平分割法（Split Horizon）
    - 当路由器向它的某个相邻路由器发送DV报文时，其中不能包含经由该相邻路由器的路径信息。
- 带触发更新的毒性逆转（Poison Reverse）法
    - 触发更新技术：一旦检测出路径崩溃，立即广播DV报文，而不必等到下一刷新周期。
    - 毒性逆转法的原理是：某路径崩溃后，最早发现的路由器将该不可达信息（距离=16）持续发送若干次。

链路状态路由算法

（LS路由算法）

链路状态路由算法

- 该算法利用**网络拓扑结构**（**链路状态**）。作如下抽象：
  - 把网络中的每个路由器看成一个节点,
  - 若两个路由器之间存在直接的连接，则它们之间存在一条边
- 这样整个网络拓扑就可抽象为一个只含有**节点**和**边**的**图**
- 参与链路状态路由算法的路由器执行如下两个操作以获得最新的网络拓扑结构：
  - **周期性地测试与相邻路由器的连接状态（邻居测试）**
  - **周期性地把链路状态广播给其它所有路由器**
- 每个路由器在获得整个网络的拓扑结构后，就可以各自独立地采用Dijkstra最短路径算法（SPF）来计算从本路由器到其它目的地的最短路径。

链路状态路由算法:基本原理

1. 每个路由器建立与它的邻接点的关系 *(“邻接”)*
2. 每个路由器周期性地产生*链路状态通报 (LSAs)* 报文并发送给**所有路由器**

   LSA = (链路 id, 链路状态,费用, 链路邻居)
3. 每个路由器维持了一个包括所有收到的LSA信息的数据库(*拓扑结构数据库* or *链路状态数据库*), 描述了带有权值的边的网络拓扑图

链路状态路由:基本原理

4. 每个路由器都使用它们的**链路状态数据库**，根据网络拓扑结构通过**最短路径算法**(Dijikstra's algorithm)来产生到每个网络的最短路径

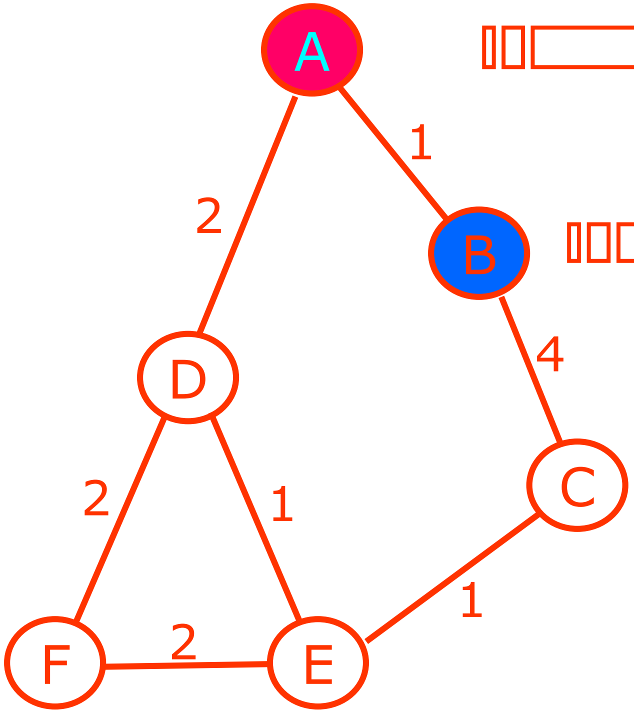
5. 路由表 = 目的地+路径的下一跳.

6. 层次路由：以降低链路状态算法的复杂性

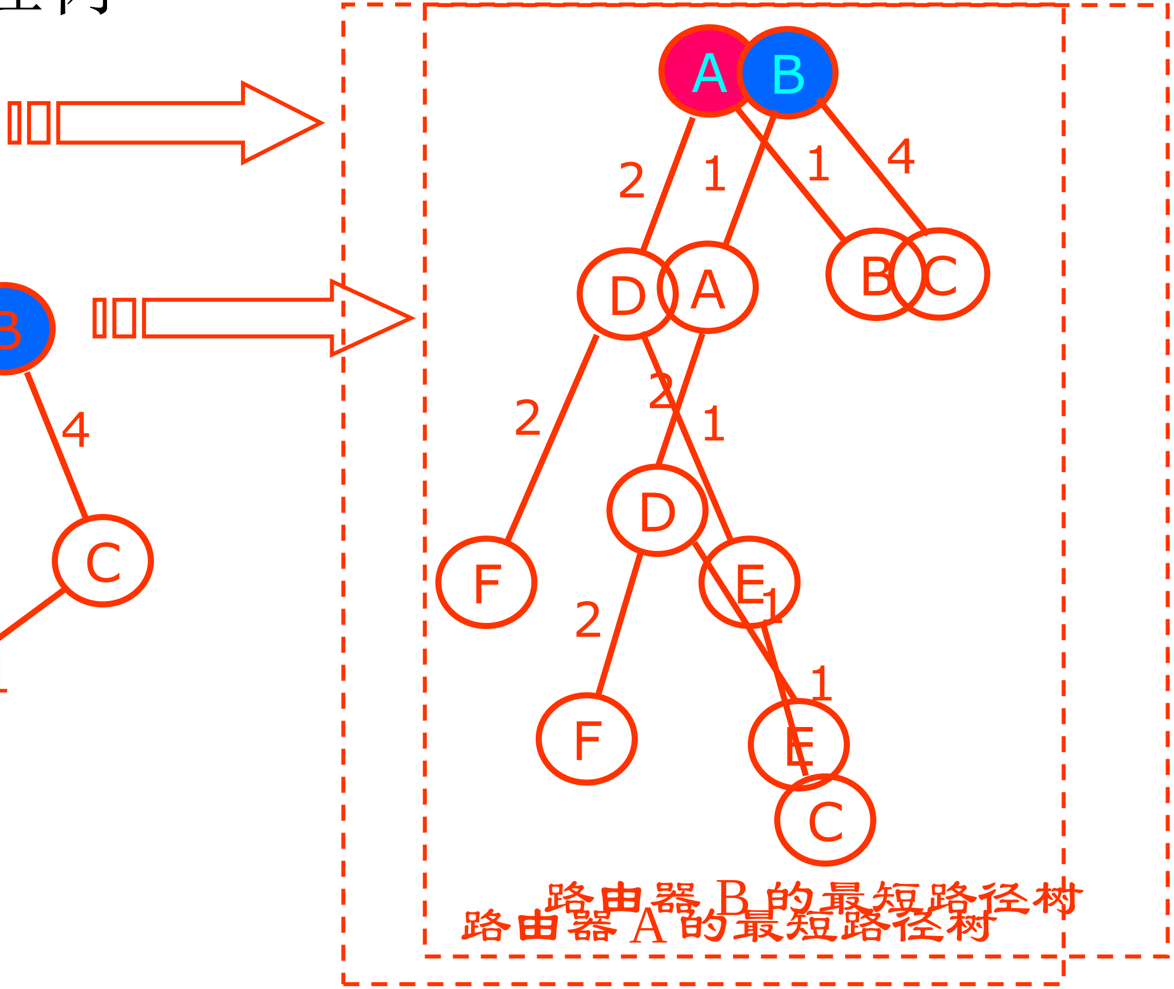
- **链路状态路由的优点**
  - □汇聚迅速
  - □网络开销小
  - □扩展性好
  - □可升级性

链路状态路由

- 生成最短路径树


A
B
C
D
E
F
1
2
4
2
1
2
1

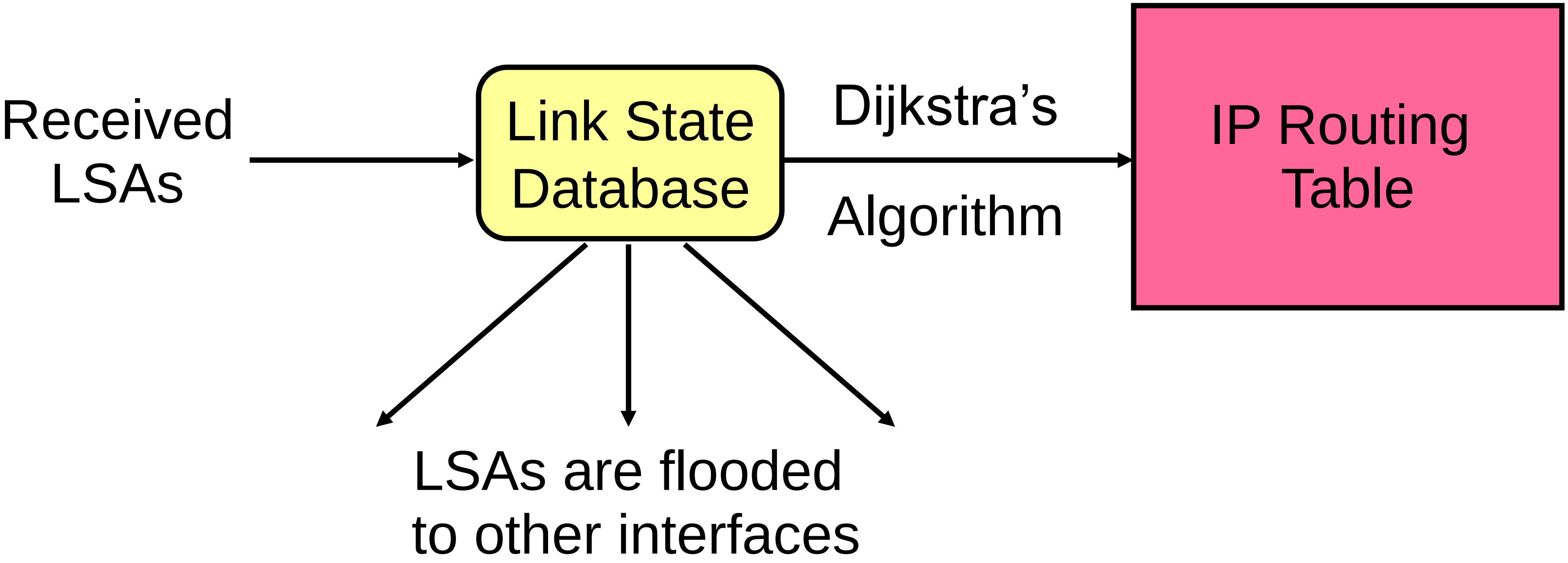
网络拓扑结构


A
B
D
A
B
C
2
1
1
4
F
D
2
2
1
F
E
2
1
E
1
C

路由器 B 的最短路径树

路由器 A 的最短路径树

Operation of a Link State Routing protocol



Received LSAs

Link State Database

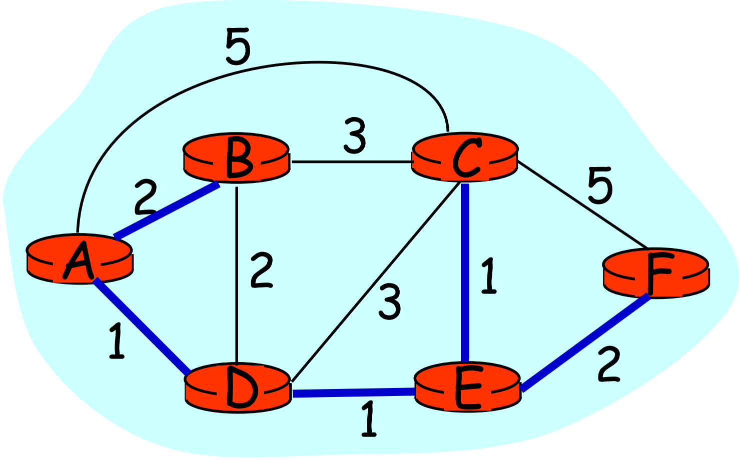
Dijkstra's Algorithm

IP Routing Table

LSAs are flooded to other interfaces

Dijkstra最短路径算法举例

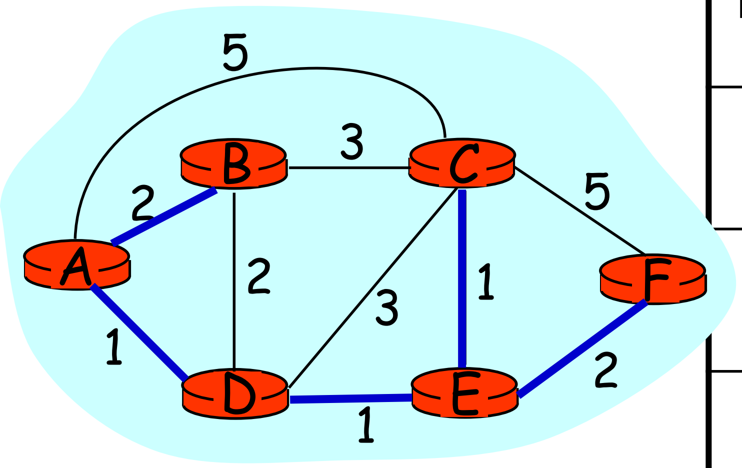
Step	set N	D(B),p(B)	D(C),p(C)	D(D),p(D)	D(E),p(E)	D(F),p(F)
0	A	2,A	5,A	1,A	infinity	infinity
1	AD	2,A	4,D		2,D	infinity
2	ADE	2,A	3,E			4,E
3	ADEB		3,E			4,E
4	ADEBC					4,E
5	ADEBCF					



以A为根的最短路径生成树叫SPF树

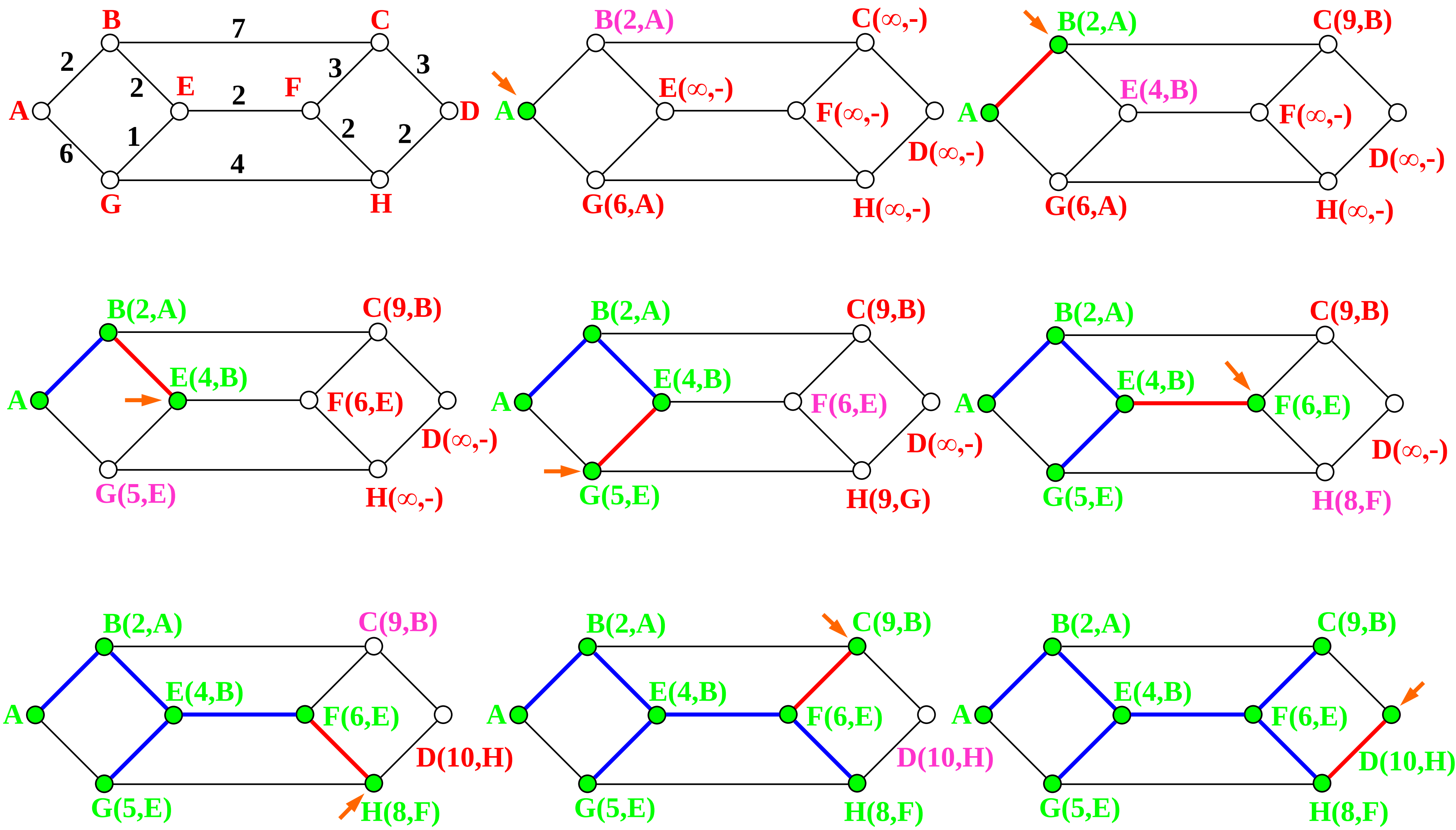
SPF: Shortest Path First

依据最短路径树生成路由表-A节点的路由表


5
3
2
2
3
1
5
1
1
2
A
B
C
D
E
F

Destination	Distance	Next-hop
B	2	--
C	3	D
D	1	--
E	2	D
F	4	D

利用Dijkstra算法求A到D的最短通路


B
7
C
2
3
2
E
2
F
3
A
D
1
2
2
6
4
G
H
B(2,A)
C(∞,-)
A
E(∞,-)
F(∞,-)
D(∞,-)
G(6,A)
H(∞,-)
B(2,A)
C(9,B)
A
E(4,B)
F(∞,-)
D(∞,-)
G(6,A)
H(∞,-)
B(2,A)
C(9,B)
E(4,B)
A
F(6,E)
D(∞,-)
G(5,E)
H(∞,-)
B(2,A)
C(9,B)
E(4,B)
A
F(6,E)
D(∞,-)
G(5,E)
H(9,G)
B(2,A)
C(9,B)
E(4,B)
A
F(6,E)
D(∞,-)
G(5,E)
H(8,F)
B(2,A)
C(9,B)
E(4,B)
A
F(6,E)
D(10,H)
G(5,E)
H(8,F)
B(2,A)
C(9,B)
E(4,B)
A
F(6,E)
D(10,H)
G(5,E)
H(8,F)
B(2,A)
C(9,B)
E(4,B)
A
F(6,E)
D(10,H)
G(5,E)
H(8,F)

最短路径为：A-B-E-F-H-D，权值为10

Dijkstra’s Algorithm

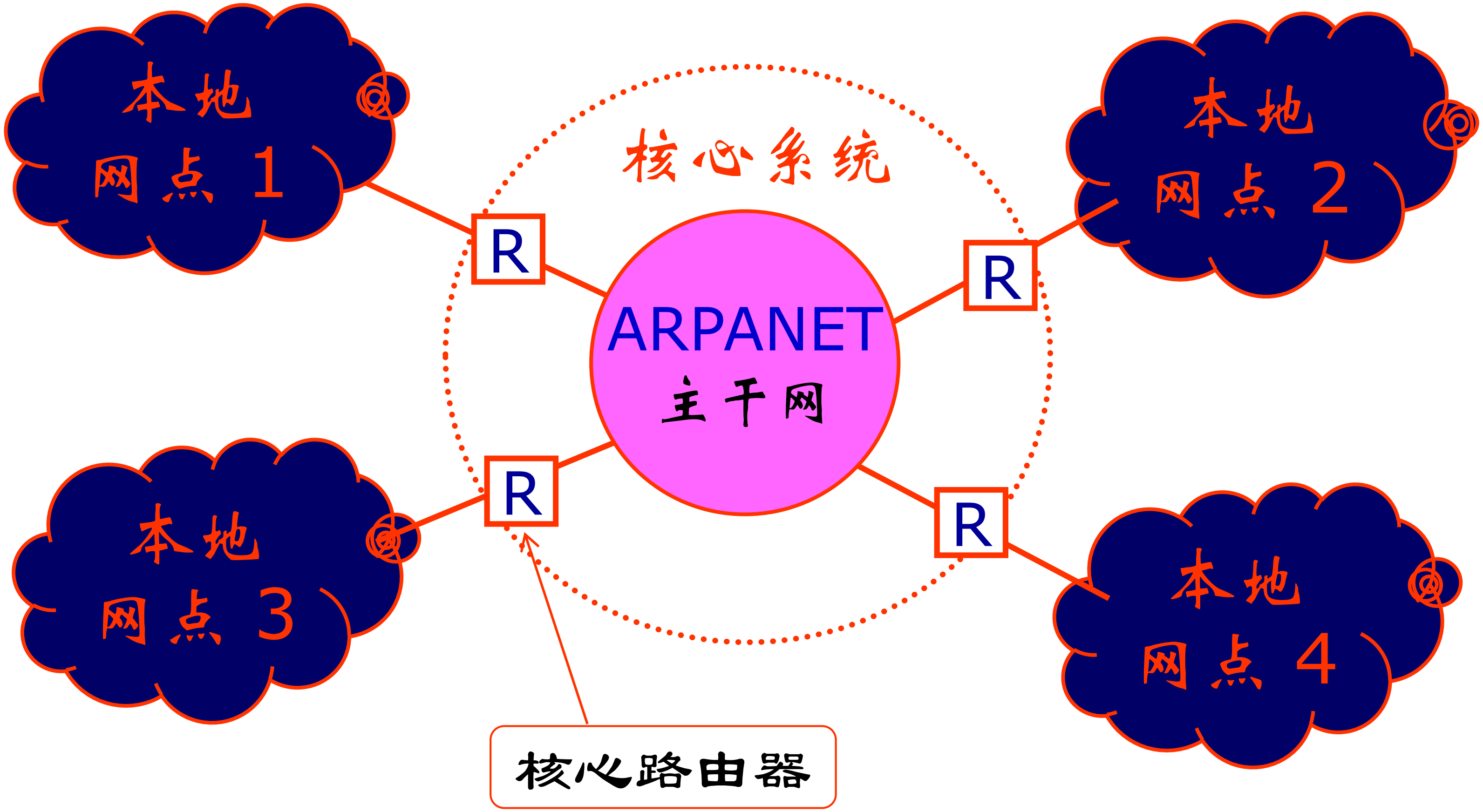
```
function Dijkstra(G, w, s)
    for each vertex v in V[G]              // Initializations
        d[v] := infinity         // Known distance function from s to v
        previous[v] := undefined
    d[s] := 0                              // Distance from s to s
    S := empty set                         // Set of all visited vertices
    Q := V[G]                              // Set of all unvisited vertices
    while Q is not an empty set            // The algorithm itself
        u := Extract_Min(Q) // Remove best vertex from priority queue
        S := S union {u}                   // Mark it 'visited'
        for each edge (u,v) outgoing from u
            if d[u] + w(u,v) < d[v]              // Relax (u,v)
                d[v] := d[u] + w(u,v)
                previous[v] := u
```

Internet

路由协议

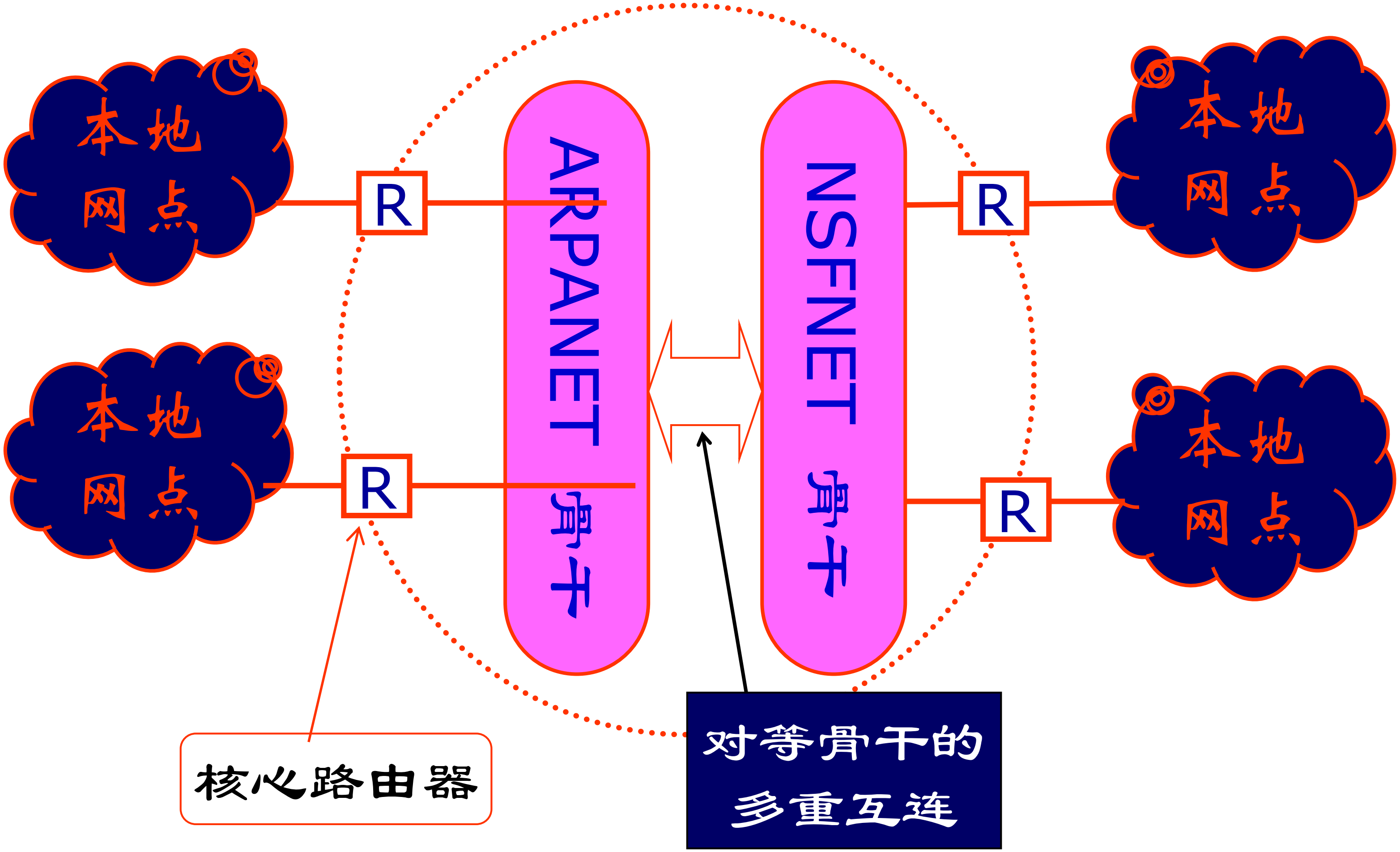
Internet的发展—拓扑结构的演变

以 ARPA 网为主干的核心结构


本地
网点 1
本地
网点 2
本地
网点 3
本地
网点 4
核心系统
R
R
R
R
ARPANET
主干网
核心路由器

Internet 的发展—拓扑结构的演变

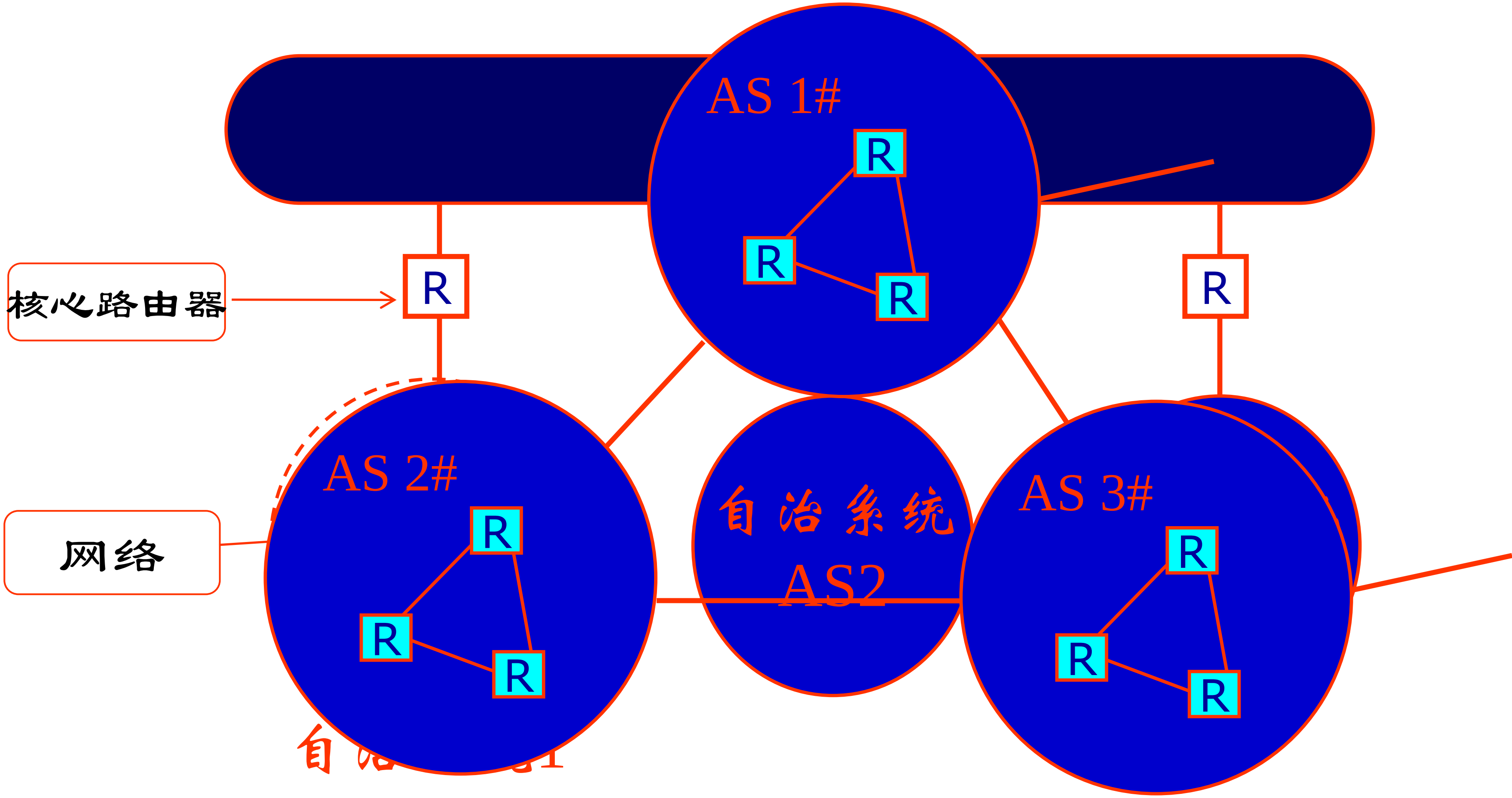
对等主干结构 —— NSFNET 的引入


本地网点
本地网点
本地网点
本地网点
R
R
R
R
ARPANET 骨干
NSFNET 骨干
核心路由器
对等骨干的多重互连

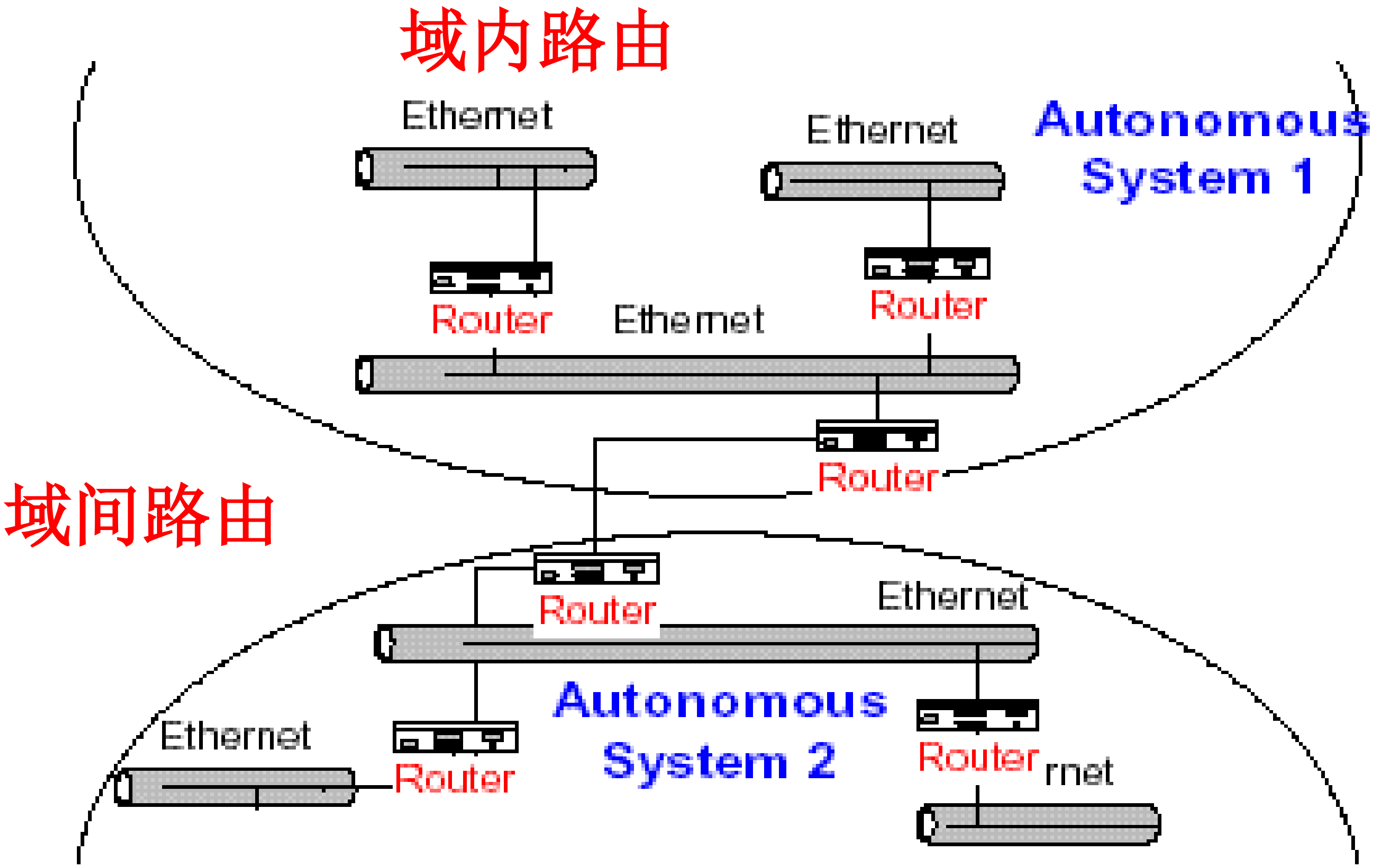
自治系统

➢ 自治系统（Autonomous System）的引入

自治系统： 由一个机构进行统一管理的网络和路由器的集合

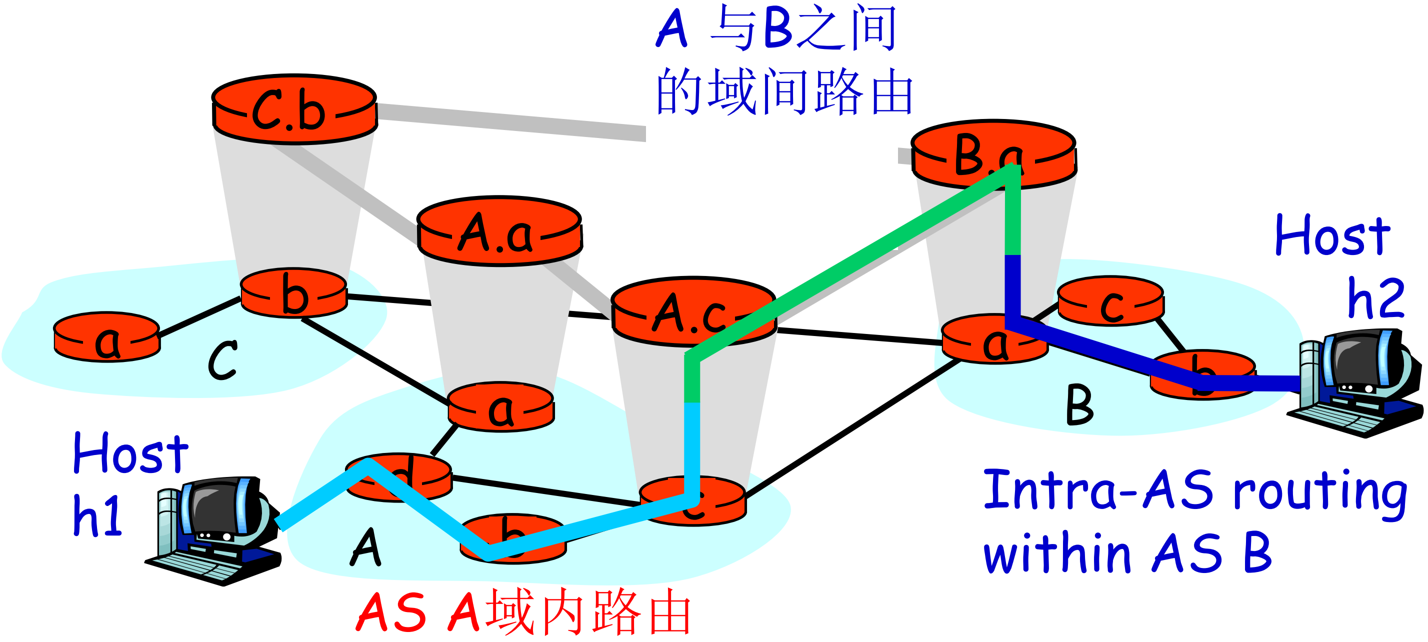

AS 1#
R
R
R
核心路由器
R
R
网络
AS 2#
R
R
R
自治系统
AS2
AS 3#
R
R
R
自 治
1

自治系统与路由


域内路由
Ethernet
Ethernet
Autonomous System 1
Router
Ethernet
Router
Router
域间路由
Router
Ethernet
Autonomous System 2
Ethernet
Router
Router
rnet

Intra-domain routing

域内和域间路由


A 与B之间
的域间路由
C.b
A.a
A.c
B.a
Host
h2
a
b
C
a
c
a
B
b
Host
h1
d
b
c
A
Intra-AS routing
within AS B
AS A域内路由

通过由几个域内和一个域间建立的路由，
一个数据报通常会经过很多跳（路由器）